WEEK 3 SQL Report

1. Introduction

Structured Query Language (SQL) is a standard language used for managing and manipulating relational databases. MySQL is an open-source relational database management system widely used in academic and industrial applications due to its reliability, performance, and ease of use. This report presents the theoretical understanding of SQL operations performed using the **MySQL Monitor (Command Line Interface)**.The MySQL Monitor provides an interactive environment where users can execute SQL commands to create databases, manage tables, insert data, and retrieve information efficiently.It provides a systematic way to create databases, define table structures, enforce data integrity constraints, and perform operations such as inserting, updating, deleting, and retrieving data. Due to its simplicity, flexibility, and powerful querying capabilities, SQL has become an essential tool in both academic learning and real-world database applications.MySQL is an open-source relational database management system that implements SQL and is extensively used across educational institutions, research environments, and industrial applications. It is known for its high performance, reliability, scalability, and strong community support. MySQL supports advanced database features such as referential integrity, indexing, constraint enforcement, and transaction management, making it suitable for developing robust and secure database-driven systems.This report focuses on the theoretical understanding of SQL operations performed using the MySQL Monitor, which is a command-line interface provided by MySQL for direct interaction with the database server. The MySQL Monitor allows users to execute SQL commands efficiently by providing immediate feedback and error messages, thereby helping in understanding the behavior and structure of SQL queries. Through this interface, users can create and manage databases, define tables with appropriate constraints, manipulate data, and perform complex queries for data retrieval and analysis. The use of the MySQL Monitor enhances conceptual clarity by offering a low-level view of database operations without the abstraction of graphical tools. It plays a significant role in strengthening fundamental SQL knowledge, improving query-writing skills, and understanding how relational databases operate internally. Hence, learning SQL through the MySQL Monitor forms a strong foundation for database design and management in both academic and professional environments.

2. MySQL Monitor Environment

The SQL operations were executed using the MySQL Monitor, which displays the message: “Welcome to the MySQL monitor. Commands end with ; or \g.” This indicates that the MySQL server is active and ready to accept SQL commands. Each command must be terminated using a semicolon or \g. The server version used is MySQL Community Server 8.0.44 (GPL), which supports advanced features such as check constraints, foreign key constraints, and cascading actions.The SQL operations were carried out using the MySQL Monitor, which is a command-line interface provided by MySQL for direct interaction with the database server. Upon successful connection, the system displays the message *“Welcome to the MySQL monitor. Commands end with ; or \g.”* This message confirms that the MySQL server is running correctly and is ready to accept and execute SQL statements from the user. The MySQL Monitor serves as a fundamental tool for understanding SQL execution by allowing users to directly input commands and observe immediate responses from the database engine.In the MySQL Monitor environment, every SQL command must be properly terminated using a semicolon (`;`) or the command delimiter (`\g`) to indicate the end of a statement. This ensures that the server correctly interprets and executes the given instruction. The interface provides clear feedback after each operation, such as confirmation messages for successful execution or descriptive error messages in case of syntax errors or constraint violations. This interactive feedback mechanism helps users identify mistakes and understand database behavior more effectively.The server version used for executing the SQL operations is **MySQL Community Server 8.0.44 (GPL)**. This version includes support for modern SQL features such as check constraints, foreign key constraints, cascading updates and deletions, and enhanced data integrity enforcement. These advanced features enable the creation of robust relational database structures that maintain consistency and accuracy of data. Using this environment allows users to gain practical exposure to industry-standard database systems while strengthening their conceptual understanding of SQL and relational database management.

3. Database Design Concept

Database design is a crucial phase in the development of any database management system, as it determines how data is organized, stored, and accessed efficiently. A well-designed database helps minimize data redundancy, ensures data integrity, and supports accurate and efficient data retrieval. Proper database design also improves system performance and simplifies future maintenance. In this experiment, the database is designed by following the fundamental principles of the relational database model, where data is represented in the form of structured tables consisting of rows and columns.The database design process begins with identifying real-world entities involved in a university system and representing them as relational tables. Entities such as departments, students, courses, and enrollments are modeled as separate tables, each created to store specific and relevant information. This separation of data into individual entities ensures clarity and prevents unnecessary duplication. Relationships among these entities are established using primary keys and foreign keys, which create logical links between tables and maintain consistency across the database. Through these relationships, related data can be easily connected and retrieved when required.Normalization techniques are applied during the design phase to further improve database efficiency. Normalization helps eliminate data redundancy and prevents anomalies during insertion, deletion, and updating of records. By dividing data into multiple related tables, the database structure becomes more organized, flexible, and easier to manage. This structured approach ensures that changes made in one table do not negatively impact other related data.In addition to normalization, various constraints such as uniqueness, not null conditions, and value checks are incorporated at the design level to enforce business rules and maintain data validity. These constraints ensure that only accurate and meaningful data is stored in the database. Overall, the database design follows a systematic and structured approach that accurately represents real-world university operations. This conceptual design forms a strong foundation for implementing SQL operations using the MySQL Monitor and enables efficient management, retrieval, and analysis of academic data within a relational database environment.

4. Tables and Relationships

Tables and relationships form the structural backbone of a relational database system. Tables are used to store data in an organized and structured manner, where each table represents a specific entity and each row corresponds to a unique record. This tabular structure allows data to be stored systematically, making it easier to manage, retrieve, and maintain. In the university database, multiple tables are created to represent key components of an academic environment, such as departments, students, courses, and enrollments. Each table is designed with carefully selected attributes to ensure clarity, consistency, and efficient data management.Relationships between tables are established using primary keys and foreign keys, which form the foundation of the relational database model. The Department table functions as a parent entity, as each department can be associated with multiple students and multiple courses. The Student table maintains a direct relationship with the Department table, ensuring that every student is linked to a valid department. This relationship helps preserve referential integrity and prevents the existence of students without an associated department. Similarly, the Course table represents academic subjects and is linked to departments, reflecting the organizational structure of courses within a university.The Enrollment table plays a critical role in the database by managing the relationship between students and courses. Since a student can enroll in multiple courses and each course can have multiple students, a many-to-many relationship exists between these entities. This relationship is resolved using the Enrollment table, which stores references to both the Student and Course tables. By doing so, the database efficiently tracks course enrollments and academic performance while avoiding data redundancy.By clearly defining tables and their relationships, the database achieves logical organization and strong data integrity. This structured design allows complex data interactions to be handled efficiently and supports meaningful data retrieval through join operations. The use of relational links ensures that the university database accurately represents real-world academic relationships and functions effectively within the MySQL database environment.

5. Integrity Constraints

Referential integrity is a fundamental concept in relational databases that ensures consistency and accuracy in relationships between tables. It is enforced using the FOREIGN KEY constraint, which establishes a dependency between a child table and a parent table. This constraint guarantees that every value in the child table's foreign key column corresponds to an existing primary key value in the parent table, thereby preventing invalid or orphaned records.Integrity constraints are fundamental rules that are automatically enforced by a database management system to ensure the accuracy, consistency, and reliability of data stored in a relational database. These constraints play a crucial role in maintaining data quality by preventing invalid, incomplete, or inconsistent information from being entered into the system. In the university database, multiple integrity constraints are implemented to ensure that data remains meaningful and logically correct throughout insertion, modification, and deletion operations.Primary key constraints are used to uniquely identify each record within a table, ensuring that every row can be distinctly referenced without ambiguity. This uniqueness allows efficient data retrieval and accurate relationship mapping between tables. Foreign key constraints are applied to establish and maintain relationships among tables, thereby enforcing referential integrity. These constraints ensure that values entered in a child table correspond to existing records in the parent table, preventing the creation of orphan records and preserving logical consistency across the database.In addition to primary and foreign key constraints, other constraints such as unique, not null, check, and default constraints further enhance data validity. Unique constraints prevent duplicate values in critical attributes such as email addresses or course codes, ensuring data uniqueness. Not null constraints ensure that mandatory fields always contain valid data and are not left empty. Check constraints restrict attribute values to specific conditions or ranges, such as limiting age or marks to acceptable values. Default constraints automatically assign predefined values when no value is provided by the user, ensuring data completeness.By enforcing these integrity constraints, the database system minimizes errors and reduces the need for manual data validation. This systematic enforcement ensures that all database operations comply with defined business rules, thereby strengthening the reliability of the university database. As a result, integrity constraints support accurate data retrieval, consistent reporting, and effective data analysis within the MySQL environment.To manage changes in parent–child relationships effectively, ON DELETE RESTRICT prevents the deletion of a parent record if dependent child records exist.ON DELETE CASCADE automatically deletes all related child records when a parent record is removed.ON UPDATE CASCADE automatically updates foreign key values in the child table when the corresponding primary key in the parent table is modified.By enforcing referential integrity, the database system maintains strong data consistency, reduces anomalies during update and deletion operations, and ensures reliable data management. This mechanism plays a crucial role in preserving the structural and logical correctness of the university database within the MySQL environment.

6. Error Handling and Database Selection in MySQL Monitor

The MySQL Monitor provides clear and descriptive error messages that help users understand mistakes made during SQL execution and guide them toward correct database operations. These error messages play an important role in ensuring that users follow proper procedures while interacting with the database. In the observed environment, repeated error messages were displayed when SQL commands were executed without selecting an active database. Although the MySQL server connection was successfully established, the absence of a selected database prevented the execution of further operations.This behavior highlights the importance of database selection as a mandatory step in SQL execution. Before performing any data definition or data manipulation operations, the database management system requires a specific database context. Without selecting a database, MySQL is unable to determine where data should be stored or retrieved, resulting in execution failure. The MySQL Monitor strictly enforces this requirement to prevent accidental modifications to unintended databases and to maintain structured database organization.The error messages generated in such situations act as immediate feedback, helping users identify the cause of failure and correct their workflow. By clearly indicating the absence of an active database, the MySQL Monitor encourages users to follow a logical sequence of operations, beginning with database selection and followed by table creation, data insertion, and other advanced operations. This feedback mechanism improves learning efficiency and reduces the chances of repeated mistakes.Additionally, the scenario demonstrates that advanced database operations such as view creation, trigger definition, audit table management, and update tracking are all dependent on an active database context. The inability to perform these operations without selecting a database reinforces the concept that logical organization in MySQL begins with proper database selection. Overall, this example illustrates how the MySQL Monitor effectively handles errors and enforces correct operational procedures, thereby improving database reliability, preventing misuse, and strengthening the user's understanding of SQL execution flow.

7. Data Manipulation Operations

Data Manipulation Operations form an essential part of SQL, as they are used to manage and modify the data stored within database tables. These operations allow users to add new records, modify existing data, and remove unwanted entries while maintaining the structure of the database. In a relational database system, data manipulation is performed without altering the table schema, ensuring that the logical design of the database remains intact.The insert operation is used to add new records into a table. It enables the database to store fresh data as and when required, such as registering new students, adding courses, or recording enrollments. Insert operations must comply with all defined constraints, including primary keys, foreign keys, and check conditions. If any constraint is violated, the database rejects the operation, thereby ensuring data validity and consistency.The update operation is used to modify existing records in a table. It allows changes to be made to specific attributes without affecting the entire dataset. Update operations are commonly used to correct data, modify academic records, or revise stored values. MySQL ensures that updates respect integrity constraints and relational dependencies, preventing invalid modifications that could lead to data inconsistency.The delete operation is used to remove records from a table when they are no longer required. Deletions are performed carefully to preserve referential integrity, especially when foreign key relationships exist between tables. Depending on the defined constraints, deleting a record may automatically remove related records or restrict the deletion to prevent data loss.Overall, data manipulation operations play a critical role in maintaining accurate, up-to-date, and meaningful information within a database. These operations enable dynamic interaction with stored data and form the foundation for effective database management in real-world applications.

7. Join Operations

Join operations in SQL are used to retrieve data from multiple tables by combining rows based on related attributes. In a relational database, data is distributed across different tables to reduce redundancy and maintain data integrity. Join operations make it possible to establish logical connections between these tables at the time of data retrieval, allowing meaningful and comprehensive information to be generated from related datasets.An inner join retrieves only those records that have matching values in both participating tables. It is commonly used when information is required only for records that exist in all related tables. This type of join is useful for generating reports where complete and consistent data is necessary, such as listing students along with their respective departments.A left join retrieves all records from the left table and the matching records from the right table. If no matching record exists in the right table, the result still includes the left table record with null values for the missing data. This join is helpful when it is important to retain all records from a primary table, even if related information is not available.A right join works in a manner opposite to the left join. It retrieves all records from the right table along with the matching records from the left table. Right joins are useful in scenarios where the focus is on preserving all records from the secondary table, such as listing all departments regardless of whether students are assigned to them.A self join is used when a table needs to be joined with itself. This type of join allows comparison of records within the same table based on a common attribute. Self joins are useful for identifying relationships or similarities among records within a single dataset.In summary, join operations are essential for effective data retrieval in relational databases. They enable the combination of data from multiple tables, support complex queries, and help present data in a meaningful and organized manner.

8.Aggregate Functions

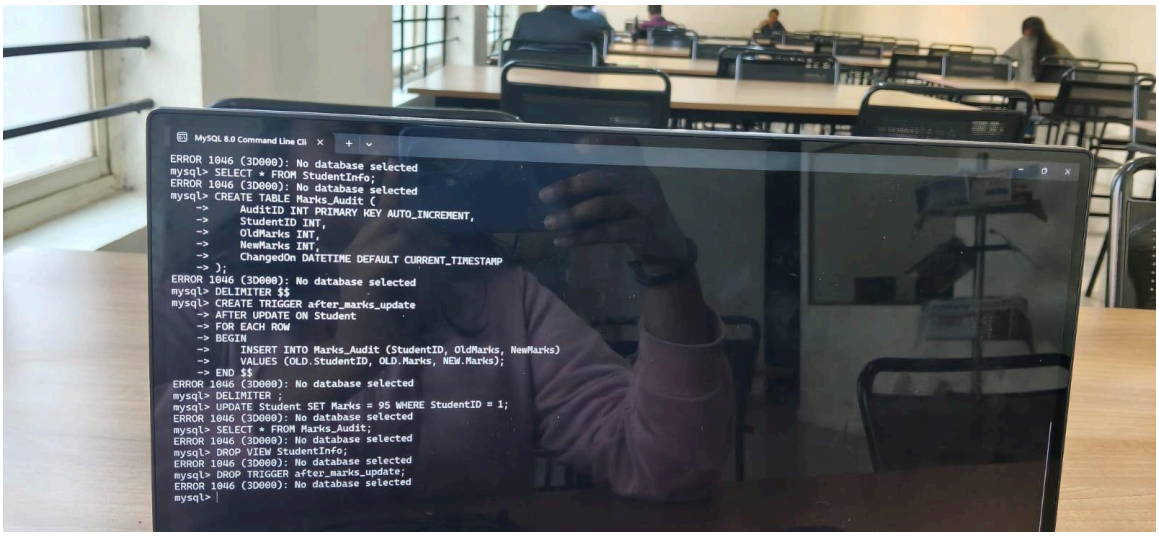
Aggregate functions are powerful tools in database management systems and programming languages used to perform calculations on a set of values and return a single summarized result. These functions operate on multiple rows of data, enabling users to derive meaningful insights quickly from large datasets. Common aggregate functions include COUNT(), which returns the number of rows matching a query; SUM(), which calculates the total sum of a numeric column; AVG(), which finds the average value; MIN() and MAX(), which identify the smallest and largest values, respectively, within a dataset.These functions are essential in data analysis and reporting, as they simplify complex queries by summarizing data points. For example, in a sales database, aggregate functions can determine total sales, average sales per region, or identify the highest and lowest sales figures. They are often used with the GROUP BY clause to perform aggregations across specific categories or groups, allowing for segmented analysis.COUNT() is useful for determining the size of a result set or counting occurrences of particular attributes. SUM() is commonly used in financial, sales, or inventory databases to find total revenue, costs, or quantities. AVG() helps analysts understand typical values such as average scores, salaries, or measurements. MIN() and MAX() pinpoint extremes like the lowest temperature, highest sales, or minimum age.When combined with the GROUP BY clause, aggregate functions calculate summaries for each category or segment separately, such as total sales by region, average test scores by class, or maximum salary by department. Additionally, the HAVING clause can filter groups based on aggregate values, such as selecting departments with average salaries above a certain threshold.Aggregate functions are vital in reporting, business intelligence, and decision-making processes. They enable quick insights that would be time-consuming to derive manually from raw data. Businesses use aggregate functions to generate monthly revenue reports, track performance metrics, and identify trends or anomalies.In summary, aggregate functions provide an efficient way to transform raw data into actionable information. Their ability to condense large volumes of data into concise statistics makes them indispensable tools in database querying, analytics, and data-driven decision making.

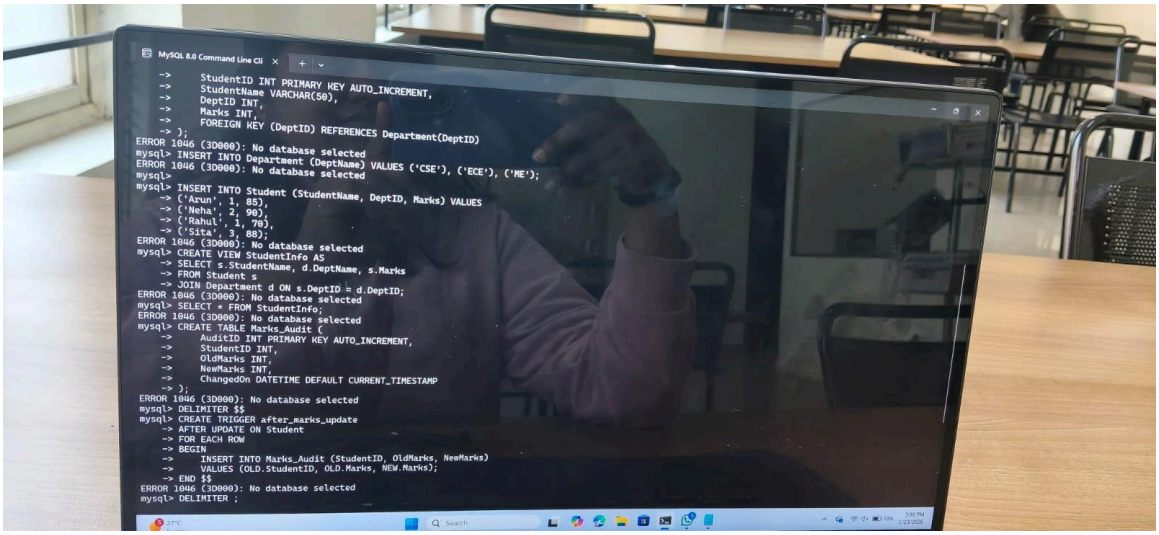
9. Triggers and views

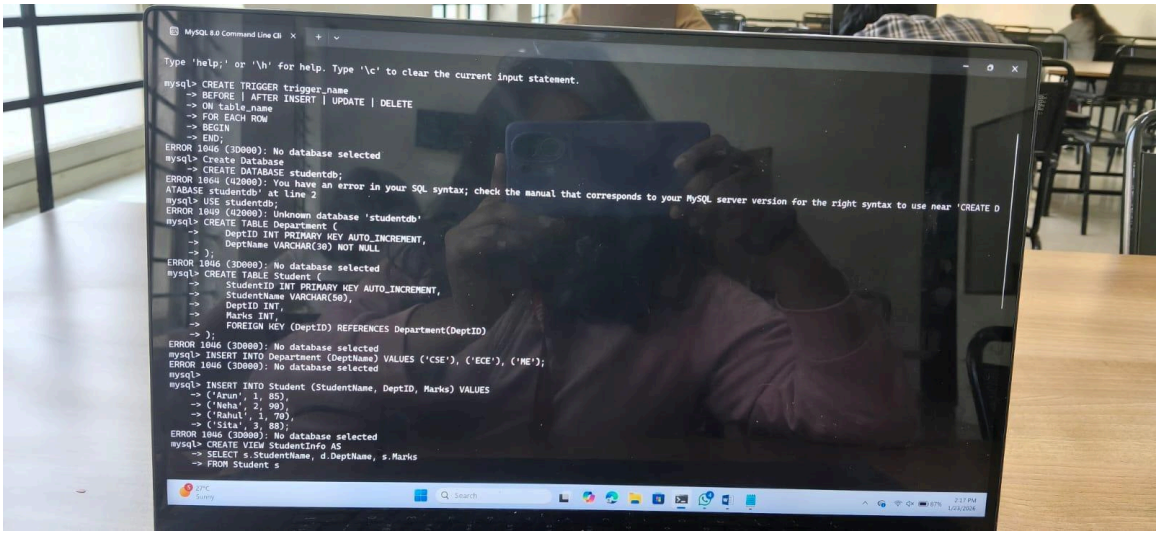
Triggers are special stored procedures that automatically execute in response to certain predefined events occurring in a database table or view. These events typically include data modification operations such as INSERT, UPDATE, and DELETE. Triggers allow developers and database administrators to embed procedural logic directly within the database, ensuring that specific actions or validations happen automatically when data changes occur.One of the main uses of triggers is to enforce business rules and data integrity constraints that go beyond what standard database constraints can achieve. For instance, a trigger can validate data before it is inserted to prevent invalid entries or automatically update related records to maintain consistency across tables. Triggers are also commonly used for auditing purposes, such as recording the time and user responsible for changes in a separate audit table.Triggers can be categorized based on their timing relative to the triggering event: BEFORE triggers run before the data modification occurs, allowing prevention of invalid operations, while AFTER triggers run after the operation, suitable for tasks like logging or cascading updates.While powerful, triggers must be designed carefully to avoid unintended consequences. Poorly implemented triggers can cause performance degradation due to additional overhead on data operations or create complex interdependencies that are hard to debug. Furthermore, because triggers operate automatically and behind the scenes, they can sometimes lead to unexpected side effects if not well documented or understood.In conclusion, triggers provide a robust mechanism to automate database behavior, enforce complex rules, and enhance data integrity, playing a critical role in maintaining reliable and consistent database systems.

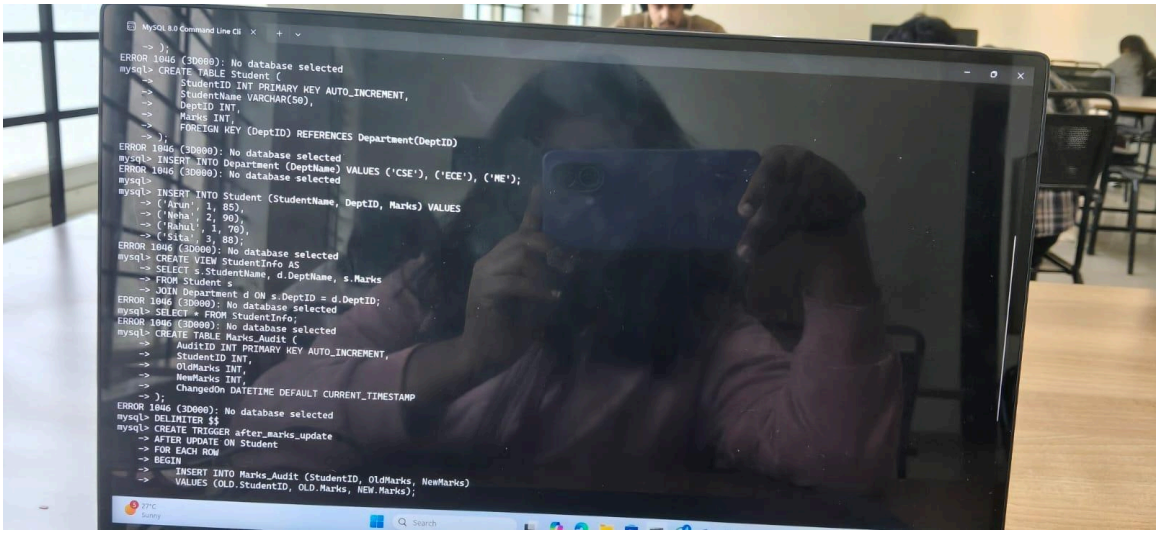
A View in a database is a virtual table that is derived from the result set of a stored SQL query. Unlike physical tables, views do not contain data themselves but dynamically display data from one or more underlying tables each time they are queried. Views serve as a powerful abstraction layer, allowing users to simplify complex data retrieval by encapsulating intricate joins, filters, and calculations into a single reusable query object. This not only enhances user productivity but also helps maintain consistency in how data is presented across different applications.Views also play a crucial role in data security and access control. By defining views that expose only specific columns or rows, database administrators can restrict user access to sensitive data without altering the underlying tables. For example, a view might show employee details without salary information to certain users. Moreover, views help enforce business logic by presenting data in a controlled, consistent manner.In addition to improving usability and security, views facilitate easier maintenance of database systems. Changes in the underlying table structures can sometimes be abstracted away by modifying views, reducing the need for widespread application-level changes. Some views, known as updatable views, allow insert, update, or delete operations, which propagate changes back to the underlying tables, although this depends on the database system and view complexity.In summary, views are essential for data abstraction, security, consistency, and simplifying complex data operations, making them invaluable components in relational database management systems.

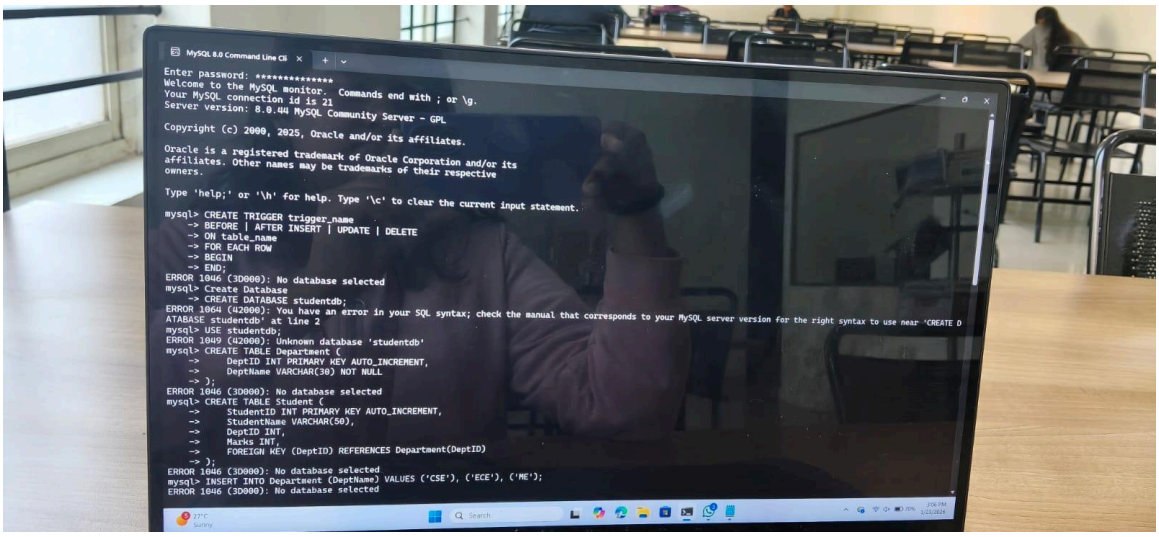
Terminals Images

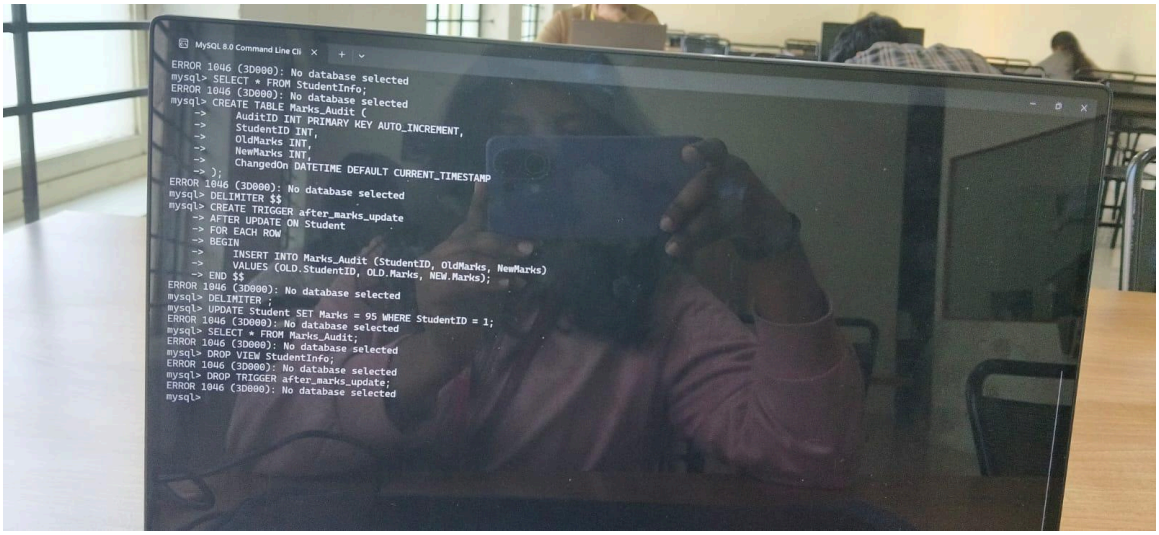


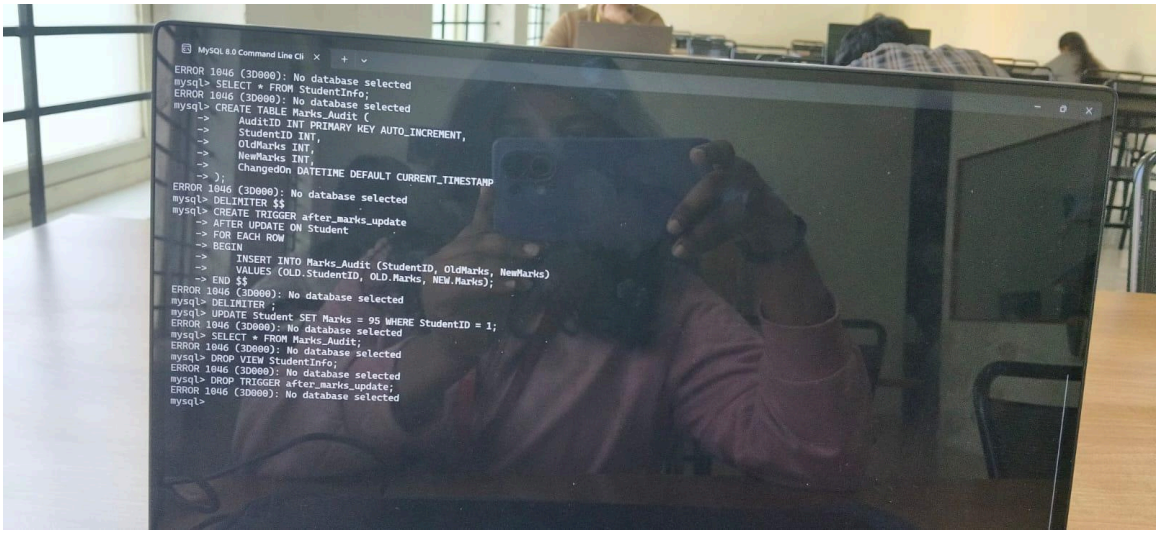


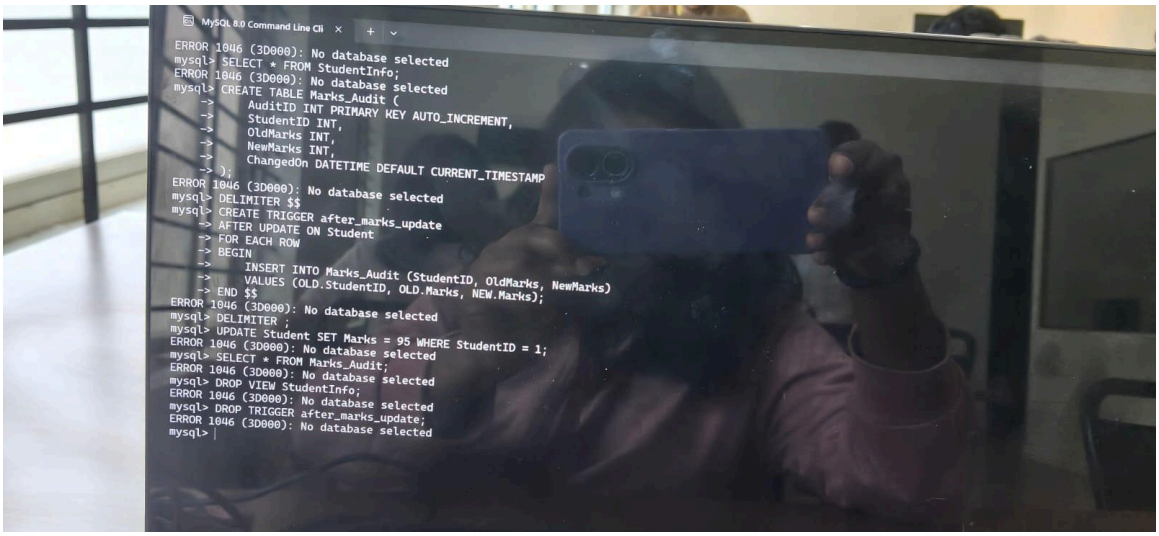


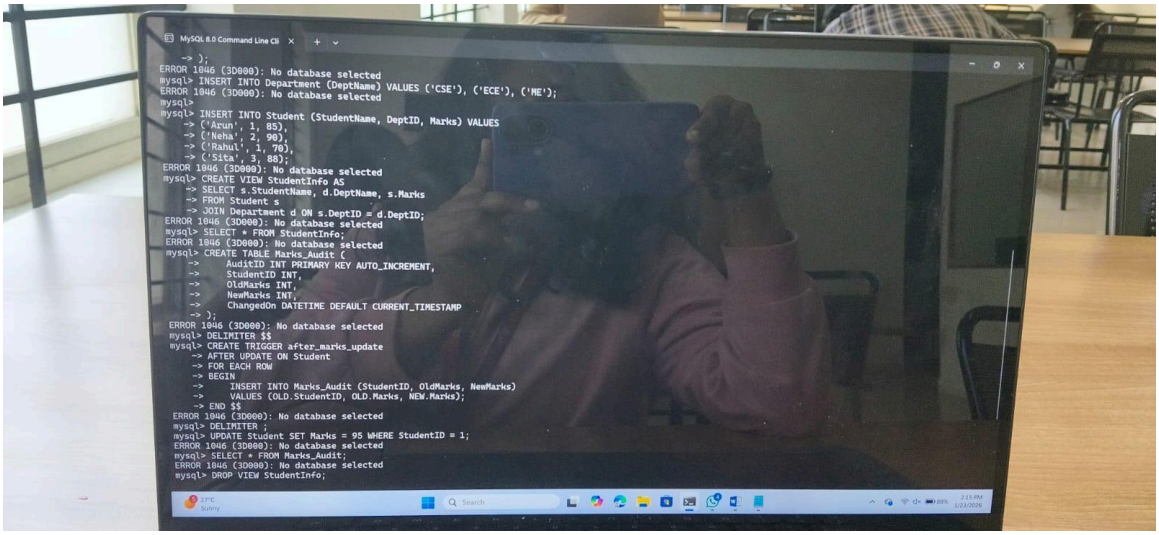

MySQL 8.0 Command Line Cli
->);
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Student (
-> StudentID INT PRIMARY KEY AUTO_INCREMENT,
-> StudentName VARCHAR(50),
-> DeptID INT,
-> Marks INT,
-> FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
->);
ERROR 1046 (3D000): No database selected
mysql> INSERT INTO Department (DeptName) VALUES ('CSE'), ('ECE'), ('ME');
ERROR 1046 (3D000): No database selected
mysql>
mysql> INSERT INTO Student (StudentName, DeptID, Marks) VALUES
-> ('Arun', 1, 85),
-> ('Neha', 2, 90),
-> ('Rahul', 1, 70),
-> ('Sita', 3, 88);
ERROR 1046 (3D000): No database selected
mysql> CREATE VIEW StudentInfo AS
-> SELECT s.StudentName, d.DeptName, s.Marks
-> FROM Student s
-> JOIN Department d ON s.DeptID = d.DeptID;
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM StudentInfo;
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Marks_Audit (
-> AuditID INT PRIMARY KEY AUTO_INCREMENT,
-> StudentID INT,
-> OldMarks INT,
-> NewMarks INT,
-> ChangedOn DATETIME DEFAULT CURRENT_TIMESTAMP
->);
ERROR 1046 (3D000): No database selected
mysql> DELIMITER $$
mysql> CREATE TRIGGER after_marks_update
-> AFTER UPDATE ON Student
-> FOR EACH ROW
-> BEGIN
-> INSERT INTO Marks_Audit (StudentID, OldMarks, NewMarks)
-> VALUES (OLD.StudentID, OLD.Marks, NEW.Marks);

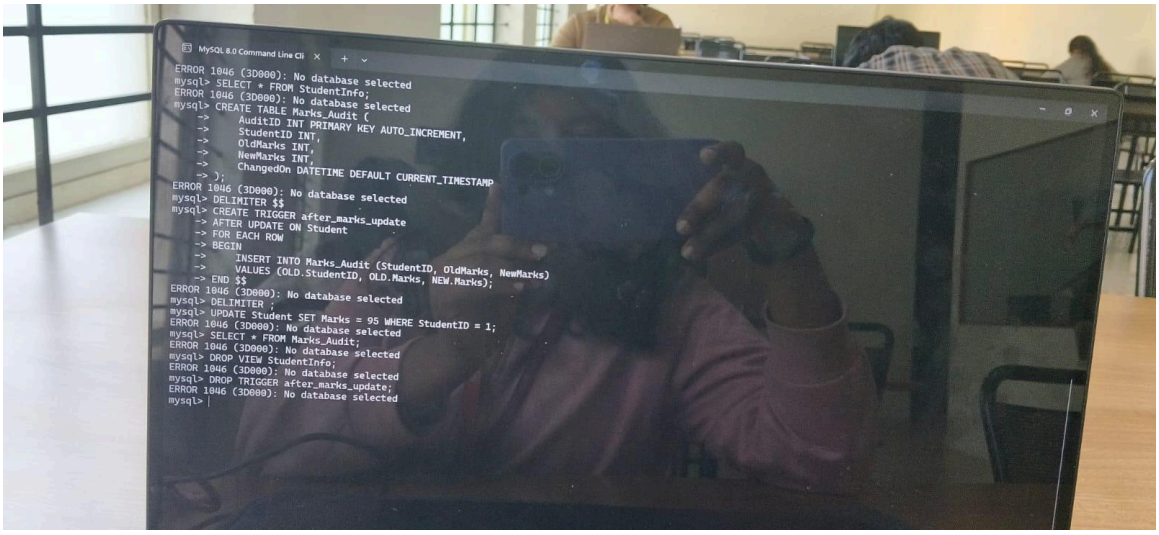

MySQL 8.0 Command Line Cli
Enter password: **********
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 21
Server version: 8.0.44 MySQL Community Server - GPL
Copyright (c) 2000, 2025, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE TRIGGER trigger_name
-> BEFORE | AFTER INSERT | UPDATE | DELETE
-> ON table_name
-> FOR EACH ROW
-> BEGIN
-> END;
ERROR 1046 (3D000): No database selected
mysql> Create Database
-> CREATE DATABASE studentdb;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'CREATE D
ATABASE studentdb' at line 2
mysql> USE studentdb;
ERROR 1049 (42000): Unknown database 'studentdb'
mysql> CREATE TABLE Department (
-> DeptID INT PRIMARY KEY AUTO_INCREMENT,
-> DeptName VARCHAR(30) NOT NULL
->);
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Student (
-> StudentID INT PRIMARY KEY AUTO_INCREMENT,
-> StudentName VARCHAR(50),
-> DeptID INT,
-> Marks INT,
-> FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
->);
ERROR 1046 (3D000): No database selected
mysql> INSERT INTO Department (DeptName) VALUES ('CSE'), ('ECE'), ('ME');
ERROR 1046 (3D000): No database selected

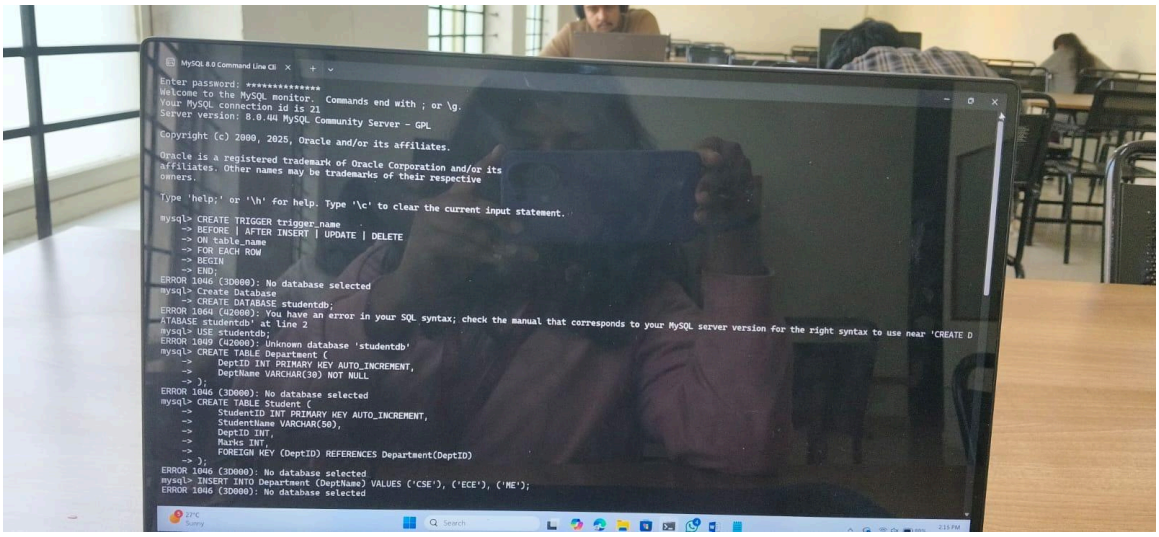

MySQL 8.0 Command Line Cli
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM StudentInfo;
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Marks_Audit (
-> AuditID INT PRIMARY KEY AUTO_INCREMENT,
-> StudentID INT,
-> OldMarks INT,
-> NewMarks INT,
-> ChangedOn DATETIME DEFAULT CURRENT_TIMESTAMP
->);
ERROR 1046 (3D000): No database selected
mysql> DELIMITER $$
mysql> CREATE TRIGGER after_marks_update
-> AFTER UPDATE ON Student
-> FOR EACH ROW
-> BEGIN
-> INSERT INTO Marks_Audit (StudentID, OldMarks, NewMarks)
-> VALUES (OLD.StudentID, OLD.Marks, NEW.Marks);
-> END $$
ERROR 1046 (3D000): No database selected
mysql> DELIMITER ;
mysql> UPDATE Student SET Marks = 95 WHERE StudentID = 1;
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM Marks_Audit;
ERROR 1046 (3D000): No database selected
mysql> DROP VIEW StudentInfo;
ERROR 1046 (3D000): No database selected
mysql> DROP TRIGGER after_marks_update;
ERROR 1046 (3D000): No database selected
mysql>

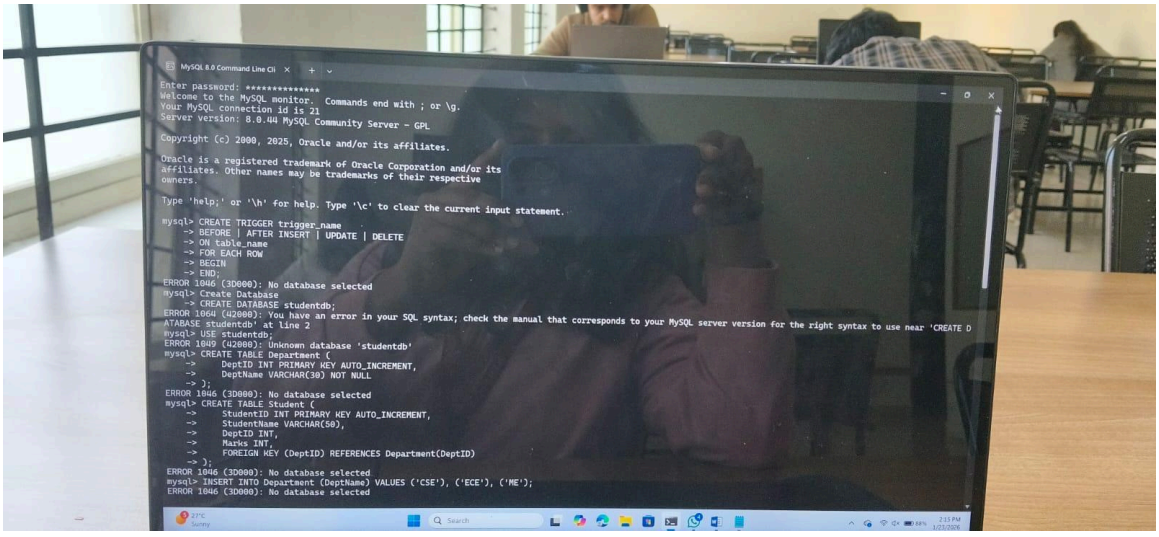

MySQL 8.0 Command Line Cli
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM StudentInfo;
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Marks_Audit (
-> AuditID INT PRIMARY KEY AUTO_INCREMENT,
-> StudentID INT,
-> OldMarks INT,
-> NewMarks INT,
-> ChangedOn DATETIME DEFAULT CURRENT_TIMESTAMP
->);
ERROR 1046 (3D000): No database selected
mysql> DELIMITER $$
mysql> CREATE TRIGGER after_marks_update
-> AFTER UPDATE ON Student
-> FOR EACH ROW
-> BEGIN
-> INSERT INTO Marks_Audit (StudentID, OldMarks, NewMarks)
-> VALUES (OLD.StudentID, OLD.Marks, NEW.Marks);
-> END $$
ERROR 1046 (3D000): No database selected
mysql> DELIMITER ;
mysql> UPDATE Student SET Marks = 95 WHERE StudentID = 1;
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM Marks_Audit;
ERROR 1046 (3D000): No database selected
mysql> DROP VIEW StudentInfo;
ERROR 1046 (3D000): No database selected
mysql> DROP TRIGGER after_marks_update;
ERROR 1046 (3D000): No database selected
mysql>

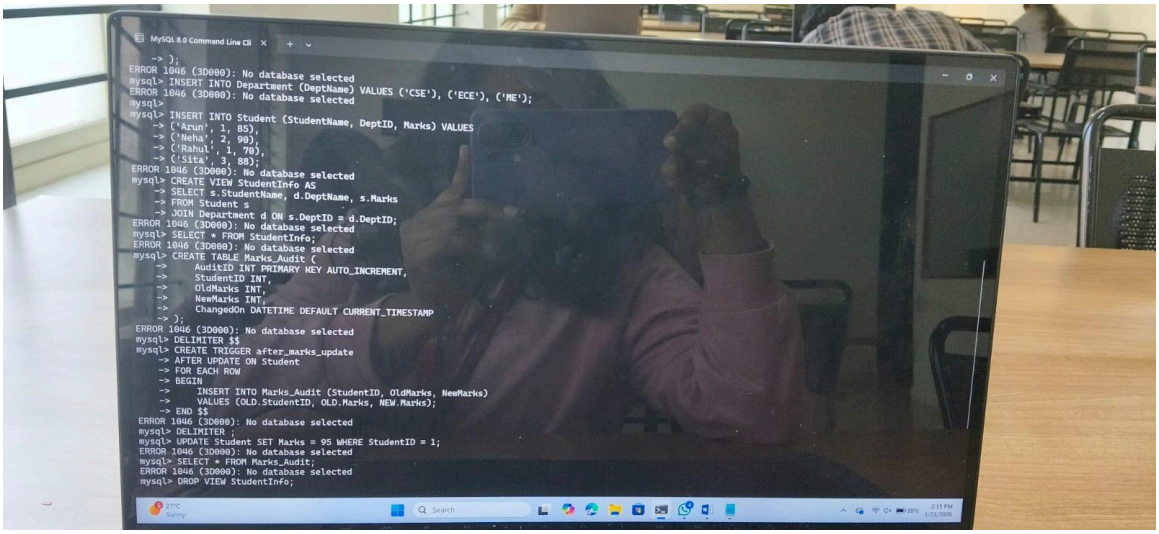

MySQL 8.0 Command Line Cli
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM StudentInfo;
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Marks_Audit (
-> AuditID INT PRIMARY KEY AUTO_INCREMENT,
-> StudentID INT,
-> OldMarks INT,
-> NewMarks INT,
-> ChangedOn DATETIME DEFAULT CURRENT_TIMESTAMP
->);
ERROR 1046 (3D000): No database selected
mysql> DELIMITER $$
mysql> CREATE TRIGGER after_marks_update
-> AFTER UPDATE ON Student
-> FOR EACH ROW
-> BEGIN
-> INSERT INTO Marks_Audit (StudentID, OldMarks, NewMarks)
-> VALUES (OLD.StudentID, OLD.Marks, NEW.Marks);
-> END $$
ERROR 1046 (3D000): No database selected
mysql> DELIMITER ;
mysql> UPDATE Student SET Marks = 95 WHERE StudentID = 1;
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM Marks_Audit;
ERROR 1046 (3D000): No database selected
mysql> DROP VIEW StudentInfo;
ERROR 1046 (3D000): No database selected
mysql> DROP TRIGGER after_marks_update;
ERROR 1046 (3D000): No database selected
mysql>

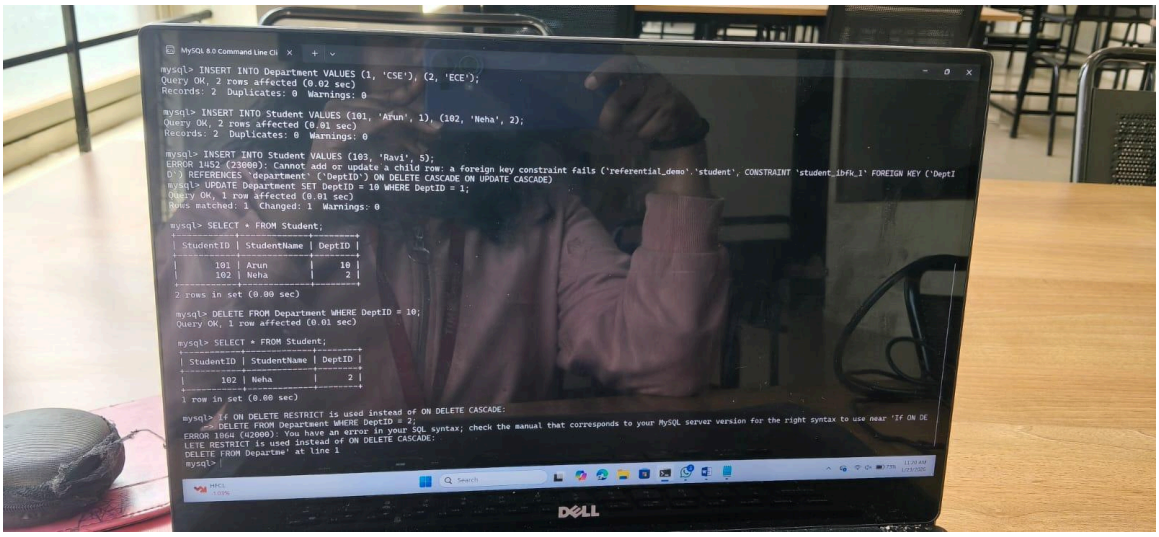

MySQL 8.0 Command Line Cli
->);
ERROR 1046 (3D000): No database selected
mysql> INSERT INTO Department (DeptName) VALUES ('CSE'), ('ECE'), ('ME');
ERROR 1046 (3D000): No database selected
mysql>
mysql> INSERT INTO Student (StudentName, DeptID, Marks) VALUES
-> ('Arun', 1, 85),
-> ('Neha', 2, 90),
-> ('Rahul', 1, 70),
-> ('Sita', 3, 88);
ERROR 1046 (3D000): No database selected
mysql> CREATE VIEW StudentInfo AS
-> SELECT s.StudentName, d.DeptName, s.Marks
-> FROM Student s
-> JOIN Department d ON s.DeptID = d.DeptID;
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM StudentInfo;
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Marks_Audit (
-> AuditID INT PRIMARY KEY AUTO_INCREMENT,
-> StudentID INT,
-> OldMarks INT,
-> NewMarks INT,
-> ChangedOn DATETIME DEFAULT CURRENT_TIMESTAMP
->);
ERROR 1046 (3D000): No database selected
mysql> DELIMITER $$
mysql> CREATE TRIGGER after_marks_update
-> AFTER UPDATE ON Student
-> FOR EACH ROW
-> BEGIN
-> INSERT INTO Marks_Audit (StudentID, OldMarks, NewMarks)
-> VALUES (OLD.StudentID, OLD.Marks, NEW.Marks);
-> END $$
ERROR 1046 (3D000): No database selected
mysql> DELIMITER ;
mysql> UPDATE Student SET Marks = 95 WHERE StudentID = 1;
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM Marks_Audit;
ERROR 1046 (3D000): No database selected
mysql> DROP VIEW StudentInfo;

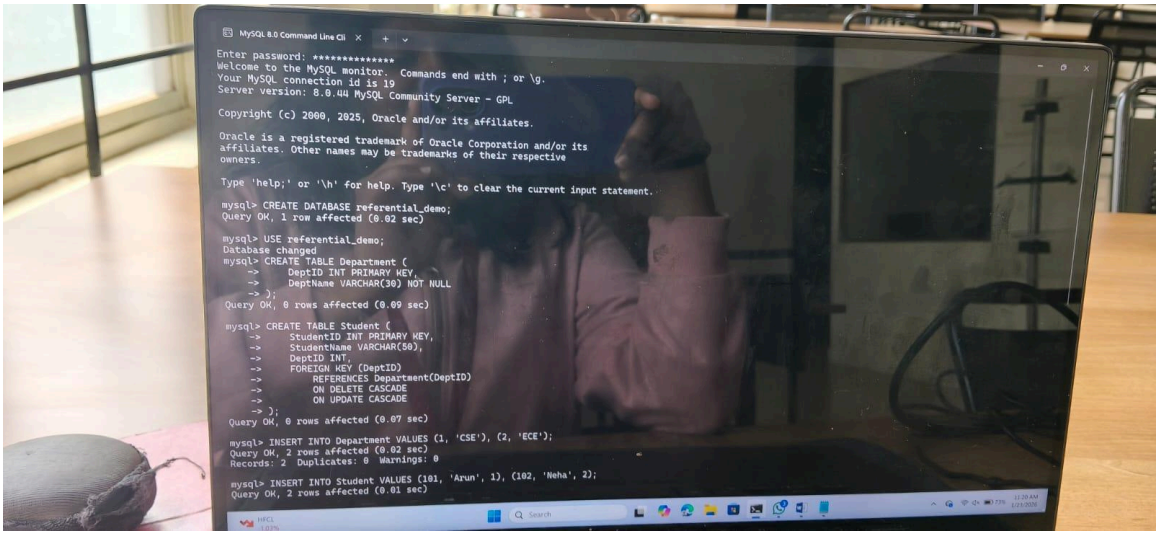

MySQL 8.0 Command Line Cli
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM StudentInfo;
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Marks_Audit (
-> AuditID INT PRIMARY KEY AUTO_INCREMENT,
-> StudentID INT,
-> OldMarks INT,
-> NewMarks INT,
-> ChangedOn DATETIME DEFAULT CURRENT_TIMESTAMP
->);
ERROR 1046 (3D000): No database selected
mysql> DELIMITER $$
mysql> CREATE TRIGGER after_marks_update
-> AFTER UPDATE ON Student
-> FOR EACH ROW
-> BEGIN
-> INSERT INTO Marks_Audit (StudentID, OldMarks, NewMarks)
-> VALUES (OLD.StudentID, OLD.Marks, NEW.Marks);
-> END $$
ERROR 1046 (3D000): No database selected
mysql> DELIMITER ;
mysql> UPDATE Student SET Marks = 95 WHERE StudentID = 1;
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM Marks_Audit;
ERROR 1046 (3D000): No database selected
mysql> DROP VIEW StudentInfo;
ERROR 1046 (3D000): No database selected
mysql> DROP TRIGGER after_marks_update;
ERROR 1046 (3D000): No database selected
mysql>

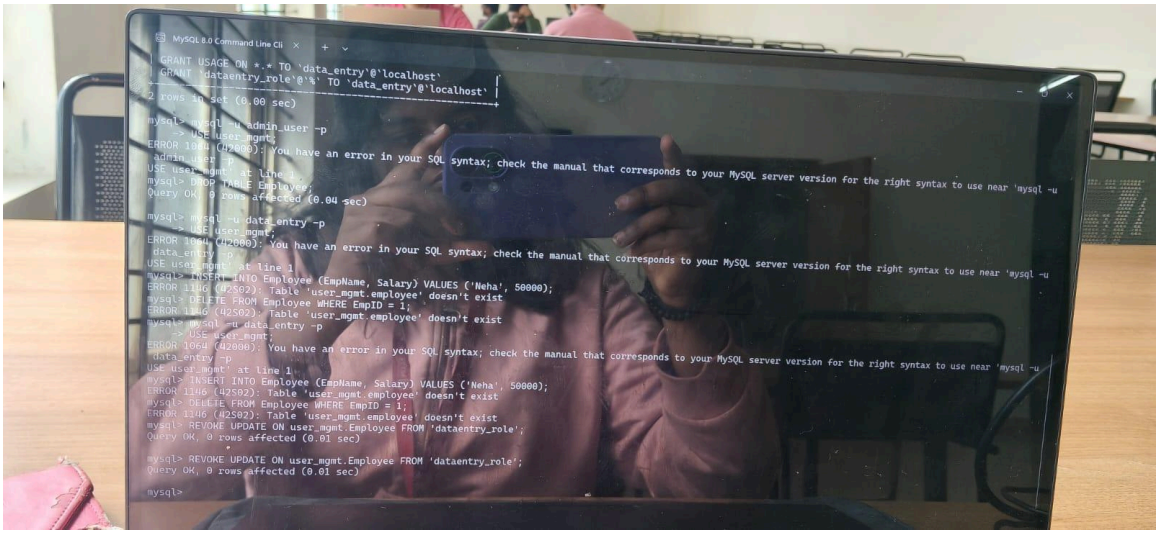

MySQL 8.0 Command Line Cli
Enter password: **********
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 21
Server version: 8.0.44 MySQL Community Server - GPL
Copyright (c) 2000, 2025, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE TRIGGER trigger_name
-> BEFORE | AFTER INSERT | UPDATE | DELETE
-> ON table_name
-> FOR EACH ROW
-> BEGIN
-> END;
ERROR 1046 (3D000): No database selected
mysql> Create Database
-> CREATE DATABASE studentdb;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'CREATE D
ATABASE studentdb' at line 2
mysql> USE studentdb;
ERROR 1049 (42000): Unknown database 'studentdb'
mysql> CREATE TABLE Department (
-> DeptID INT PRIMARY KEY AUTO_INCREMENT,
-> DeptName VARCHAR(30) NOT NULL
->);
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Student (
-> StudentID INT PRIMARY KEY AUTO_INCREMENT,
-> StudentName VARCHAR(50),
-> DeptID INT,
-> Marks INT,
-> FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
->);
ERROR 1046 (3D000): No database selected
mysql> INSERT INTO Department (DeptName) VALUES ('CSE'), ('ECE'), ('ME');
ERROR 1046 (3D000): No database selected

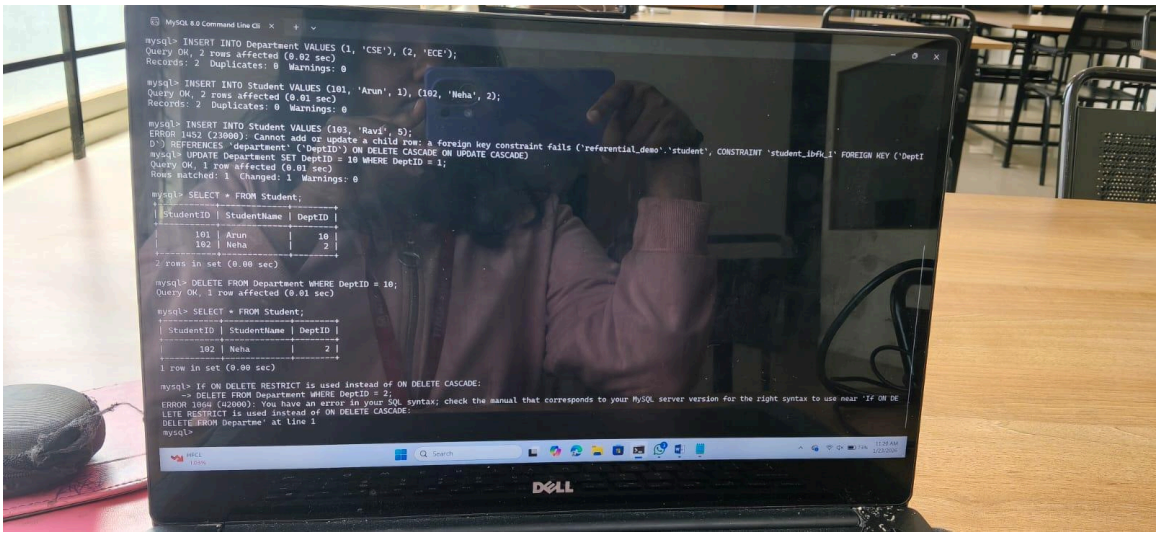

MySQL 8.0 Command Line Cli
Enter password: **********
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 21
Server version: 8.0.44 MySQL Community Server - GPL
Copyright (c) 2000, 2025, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE TRIGGER trigger_name
-> BEFORE | AFTER INSERT | UPDATE | DELETE
-> ON table_name
-> FOR EACH ROW
-> BEGIN
-> END;
ERROR 1046 (3D000): No database selected
mysql> Create Database
-> CREATE DATABASE studentdb;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'CREATE D
ATABASE studentdb' at line 2
mysql> USE studentdb;
ERROR 1049 (42000): Unknown database 'studentdb'
mysql> CREATE TABLE Department (
-> DeptID INT PRIMARY KEY AUTO_INCREMENT,
-> DeptName VARCHAR(30) NOT NULL
->);
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Student (
-> StudentID INT PRIMARY KEY AUTO_INCREMENT,
-> StudentName VARCHAR(50),
-> DeptID INT,
-> Marks INT,
-> FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
->);
ERROR 1046 (3D000): No database selected
mysql> INSERT INTO Department (DeptName) VALUES ('CSE'), ('ECE'), ('ME');
ERROR 1046 (3D000): No database selected

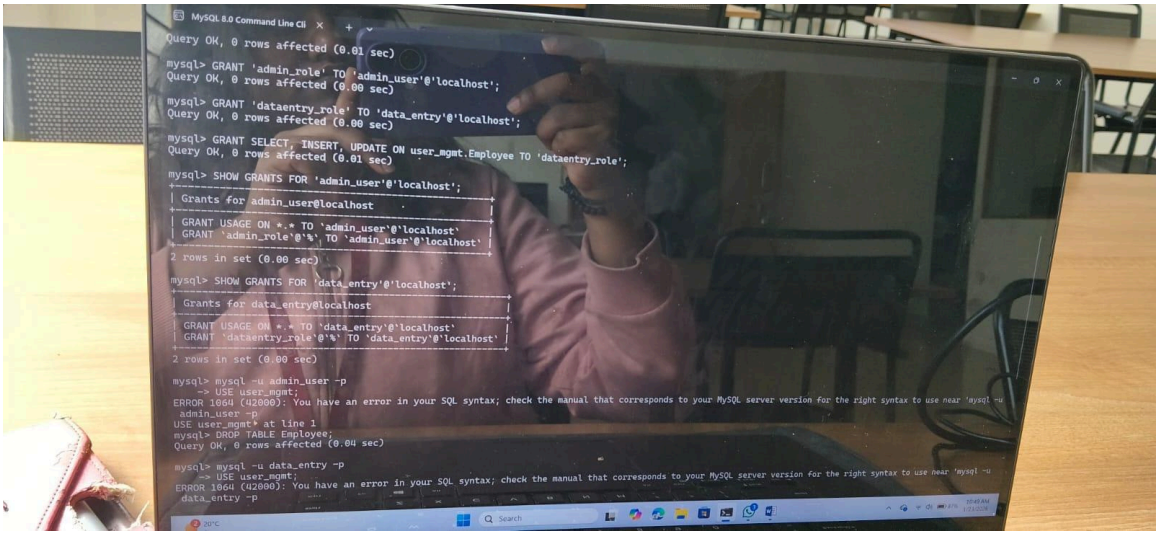

MySQL 8.0 Command Line Cli
->);
ERROR 1046 (3D000): No database selected
mysql> INSERT INTO Department (DeptName) VALUES ('CSE'), ('ECE'), ('ME');
ERROR 1046 (3D000): No database selected
mysql>
mysql> INSERT INTO Student (StudentName, DeptID, Marks) VALUES
-> ('Arun', 1, 85),
-> ('Neha', 2, 90),
-> ('Rahul', 1, 70),
-> ('Sita', 3, 88);
ERROR 1046 (3D000): No database selected
mysql> CREATE VIEW StudentInfo AS
-> SELECT s.StudentName, d.DeptName, s.Marks
-> FROM Student s
-> JOIN Department d ON s.DeptID = d.DeptID;
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM StudentInfo;
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Marks_Audit (
-> AuditID INT PRIMARY KEY AUTO_INCREMENT,
-> StudentID INT,
-> OldMarks INT,
-> NewMarks INT,
-> ChangedOn DATETIME DEFAULT CURRENT_TIMESTAMP
->);
ERROR 1046 (3D000): No database selected
mysql> DELIMITER $$
mysql> CREATE TRIGGER after_marks_update
-> AFTER UPDATE ON Student
-> FOR EACH ROW
-> BEGIN
-> INSERT INTO Marks_Audit (StudentID, OldMarks, NewMarks)
-> VALUES (OLD.StudentID, OLD.Marks, NEW.Marks);
-> END $$
ERROR 1046 (3D000): No database selected
mysql> DELIMITER ;
mysql> UPDATE Student SET Marks = 95 WHERE StudentID = 1;
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM Marks_Audit;
ERROR 1046 (3D000): No database selected
mysql> DROP VIEW StudentInfo;

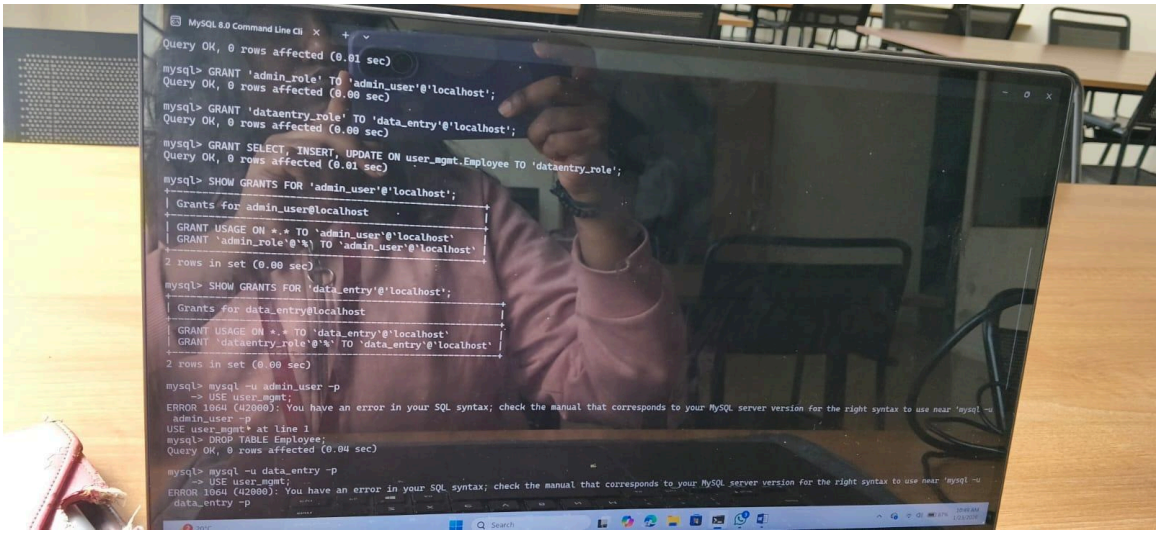

MySQL 8.0 Command Line Cli
mysql> INSERT INTO Department VALUES (1, 'CSE'), (2, 'ECE');
Query OK, 2 rows affected (0.02 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql> INSERT INTO Student VALUES (101, 'Arun', 1), (102, 'Neha', 2);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql> INSERT INTO Student VALUES (103, 'Ravi', 5);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`referential_demo`.`student`, CONSTRAINT `student_ibfk_1` FOREIGN KEY (`DeptI
D`) REFERENCES `department` (`DeptID`) ON DELETE CASCADE ON UPDATE CASCADE)
mysql> UPDATE Department SET DeptID = 10 WHERE DeptID = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0
mysql> SELECT * FROM Student;
+-----------+-------------+--------+
| StudentID | StudentName | DeptID |
+-----------+-------------+--------+
| 101 | Arun | 10 |
| 102 | Neha | 2 |
+-----------+-------------+--------+
2 rows in set (0.00 sec)
mysql> DELETE FROM Department WHERE DeptID = 10;
Query OK, 1 row affected (0.01 sec)
mysql> SELECT * FROM Student;
+-----------+-------------+--------+
| StudentID | StudentName | DeptID |
+-----------+-------------+--------+
| 102 | Neha | 2 |
+-----------+-------------+--------+
1 row in set (0.00 sec)
mysql> If ON DELETE RESTRICT is used instead of ON DELETE CASCADE:
-> DELETE FROM Department WHERE DeptID = 2;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'If ON DE
LETE RESTRICT is used instead of ON DELETE CASCADE:
DELETE FROM Departme' at line 1
mysql>
DELL

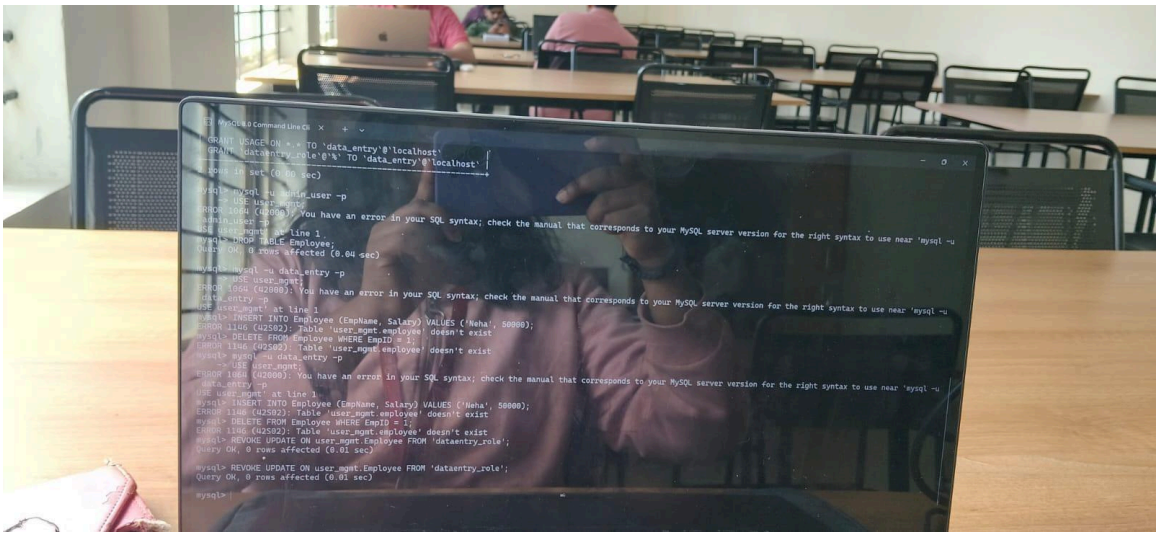

MySQL 8.0 Command Line Cli
Enter password: **********
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 19
Server version: 8.0.44 MySQL Community Server - GPL
Copyright (c) 2000, 2025, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE DATABASE referential_demo;
Query OK, 1 row affected (0.02 sec)
mysql> USE referential_demo;
Database changed
mysql> CREATE TABLE Department (
-> DeptID INT PRIMARY KEY,
-> DeptName VARCHAR(30) NOT NULL
->);
Query OK, 0 rows affected (0.09 sec)
mysql> CREATE TABLE Student (
-> StudentID INT PRIMARY KEY,
-> StudentName VARCHAR(50),
-> DeptID INT,
-> FOREIGN KEY (DeptID)
-> REFERENCES Department(DeptID)
-> ON DELETE CASCADE
-> ON UPDATE CASCADE
->);
Query OK, 0 rows affected (0.07 sec)
mysql> INSERT INTO Department VALUES (1, 'CSE'), (2, 'ECE');
Query OK, 2 rows affected (0.02 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql> INSERT INTO Student VALUES (101, 'Arun', 1), (102, 'Neha', 2);
Query OK, 2 rows affected (0.01 sec)

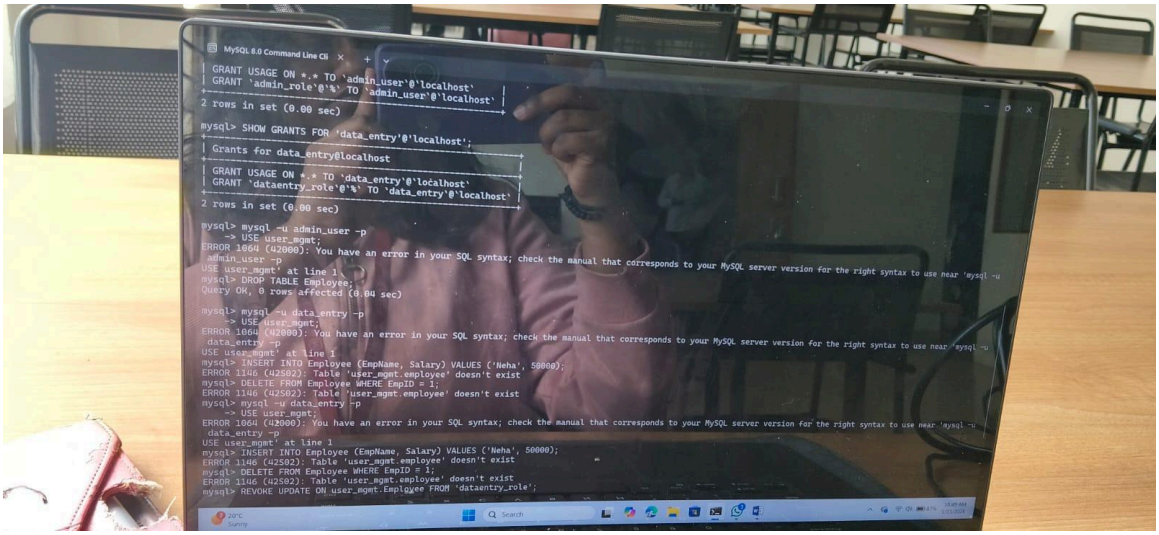

MySQL 8.0 Command Line Cli
GRANT USAGE ON *.* TO 'data_entry'@'localhost'
GRANT 'dataentry_role'@'%' TO 'data_entry'@'localhost'
2 rows in set (0.00 sec)
mysql> mysql -u admin_user -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u admin_user -p
USE user_mgmt' at line 1
mysql> DROP TABLE Employee;
Query OK, 0 rows affected (0.04 sec)
mysql> mysql -u data_entry -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u data_entry -p
USE user_mgmt' at line 1
mysql> INSERT INTO Employee (EmpName, Salary) VALUES ('Neha', 50000);
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> DELETE FROM Employee WHERE EmpID = 1;
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> mysql -u data_entry -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u data_entry -p
USE user_mgmt' at line 1
mysql> INSERT INTO Employee (EmpName, Salary) VALUES ('Neha', 50000);
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> DELETE FROM Employee WHERE EmpID = 1;
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> REVOKE UPDATE ON user_mgmt.Employee FROM 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql> REVOKE UPDATE ON user_mgmt.Employee FROM 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql>

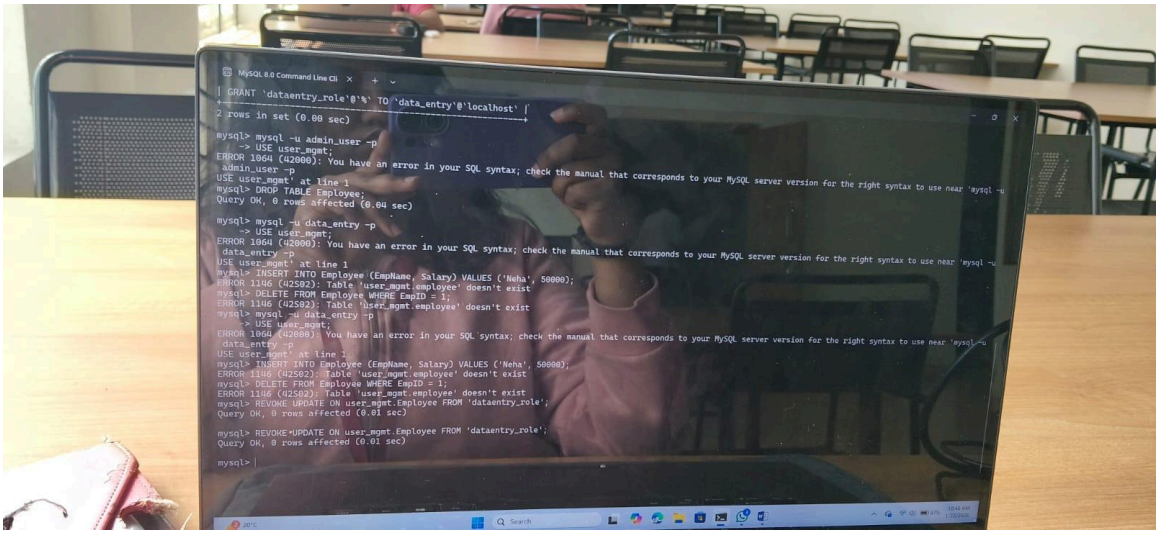

MySQL 8.0 Command Line Cli
mysql> INSERT INTO Department VALUES (1, 'CSE'), (2, 'ECE');
Query OK, 2 rows affected (0.02 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql> INSERT INTO Student VALUES (101, 'Arun', 1), (102, 'Neha', 2);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql> INSERT INTO Student VALUES (103, 'Ravi', 5);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`referential_demo`.`student`, CONSTRAINT `student_ibfk_1` FOREIGN KEY (`DeptI
D`) REFERENCES `department` (`DeptID`) ON DELETE CASCADE ON UPDATE CASCADE)
mysql> UPDATE Department SET DeptID = 10 WHERE DeptID = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0
mysql> SELECT * FROM Student;
+-----------+-------------+--------+
| StudentID | StudentName | DeptID |
+-----------+-------------+--------+
| 101 | Arun | 10 |
| 102 | Neha | 2 |
+-----------+-------------+--------+
2 rows in set (0.00 sec)
mysql> DELETE FROM Department WHERE DeptID = 10;
Query OK, 1 row affected (0.01 sec)
mysql> SELECT * FROM Student;
+-----------+-------------+--------+
| StudentID | StudentName | DeptID |
+-----------+-------------+--------+
| 102 | Neha | 2 |
+-----------+-------------+--------+
1 row in set (0.00 sec)
mysql> If ON DELETE RESTRICT is used instead of ON DELETE CASCADE:
-> DELETE FROM Department WHERE DeptID = 2;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'If ON DE
LETE RESTRICT is used instead of ON DELETE CASCADE:
DELETE FROM Departme' at line 1
mysql>
DELL

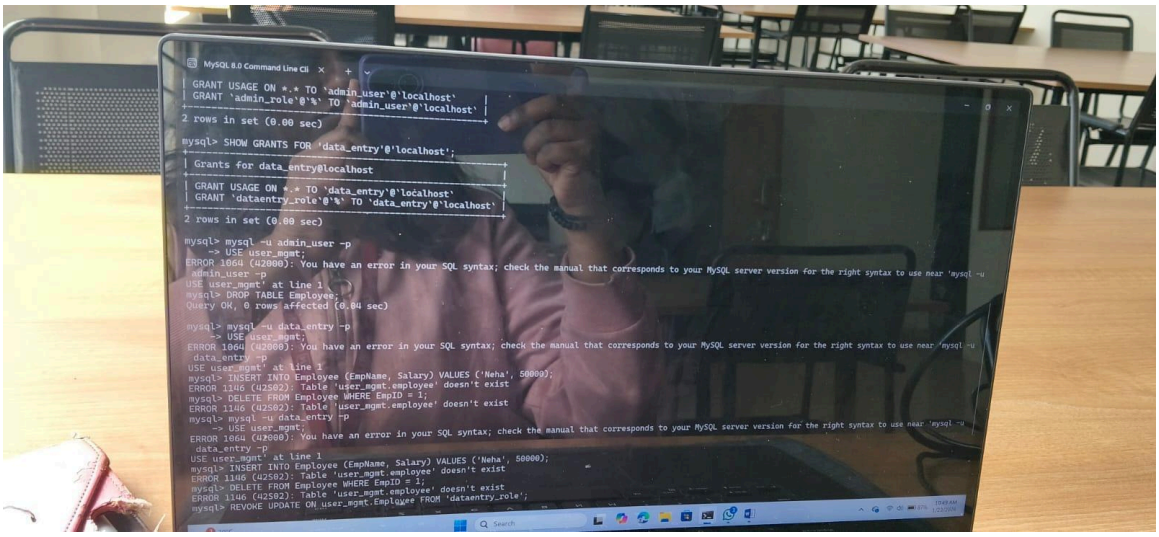

MySQL 8.0 Command Line Cli
Query OK, 0 rows affected (0.01 sec)
mysql> GRANT 'admin_role' TO 'admin_user'@'localhost';
Query OK, 0 rows affected (0.00 sec)
mysql> GRANT 'dataentry_role' TO 'data_entry'@'localhost';
Query OK, 0 rows affected (0.00 sec)
mysql> GRANT SELECT, INSERT, UPDATE ON user_mgmt.Employee TO 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql> SHOW GRANTS FOR 'admin_user'@'localhost';
+--+
| Grants for admin_user@localhost |
+--+
| GRANT USAGE ON *.* TO `admin_user`@`localhost` |
| GRANT `admin_role`@`%` TO `admin_user`@`localhost` |
+--+
2 rows in set (0.00 sec)
mysql> SHOW GRANTS FOR 'data_entry'@'localhost';
+--+
| Grants for data_entry@localhost |
+--+
| GRANT USAGE ON *.* TO `data_entry`@`localhost` |
| GRANT `dataentry_role`@`%` TO `data_entry`@`localhost` |
+--+
2 rows in set (0.00 sec)
mysql> mysql -u admin_user -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u admin_user -p
USE user_mgmt' at line 1
mysql> DROP TABLE Employee;
Query OK, 0 rows affected (0.04 sec)
mysql> mysql -u data_entry -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u data_entry -p

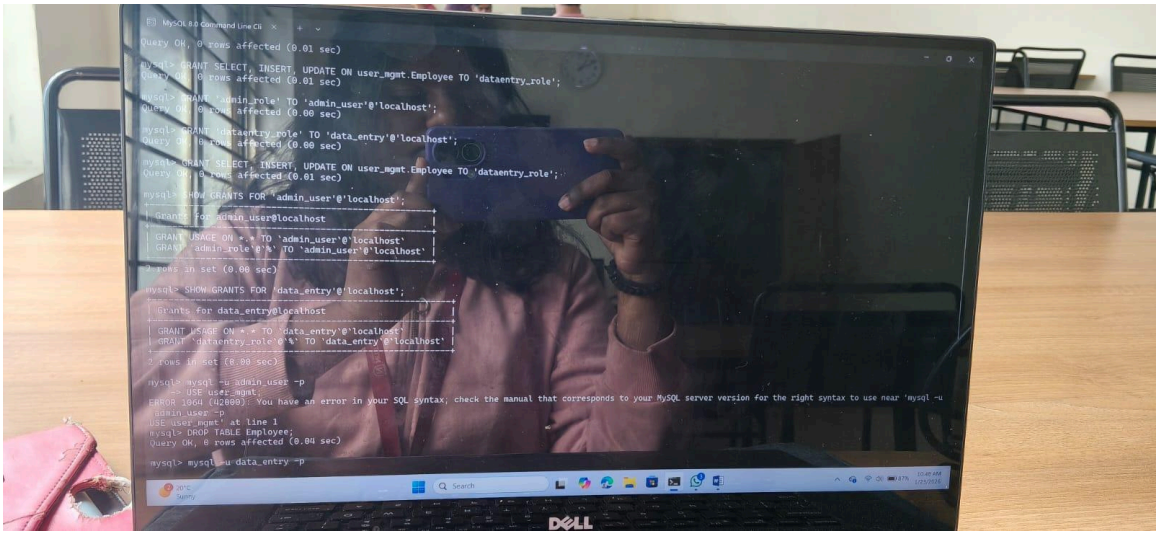

MySQL 8.0 Command Line Cli
Query OK, 0 rows affected (0.01 sec)
mysql> GRANT 'admin_role' TO 'admin_user'@'localhost';
Query OK, 0 rows affected (0.00 sec)
mysql> GRANT 'dataentry_role' TO 'data_entry'@'localhost';
Query OK, 0 rows affected (0.00 sec)
mysql> GRANT SELECT, INSERT, UPDATE ON user_mgmt.Employee TO 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql> SHOW GRANTS FOR 'admin_user'@'localhost';
Grants for admin_user@localhost
GRANT USAGE ON *.* TO `admin_user`@`localhost`
GRANT `admin_role`@`%` TO `admin_user`@`localhost`
2 rows in set (0.00 sec)
mysql> SHOW GRANTS FOR 'data_entry'@'localhost';
Grants for data_entry@localhost
GRANT USAGE ON *.* TO `data_entry`@`localhost`
GRANT `dataentry_role`@`%` TO `data_entry`@`localhost`
2 rows in set (0.00 sec)
mysql> mysql -u admin_user -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u admin_user -p
USE user_mgmt' at line 1
mysql> DROP TABLE Employee;
Query OK, 0 rows affected (0.04 sec)
mysql> mysql -u data_entry -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u data_entry -p

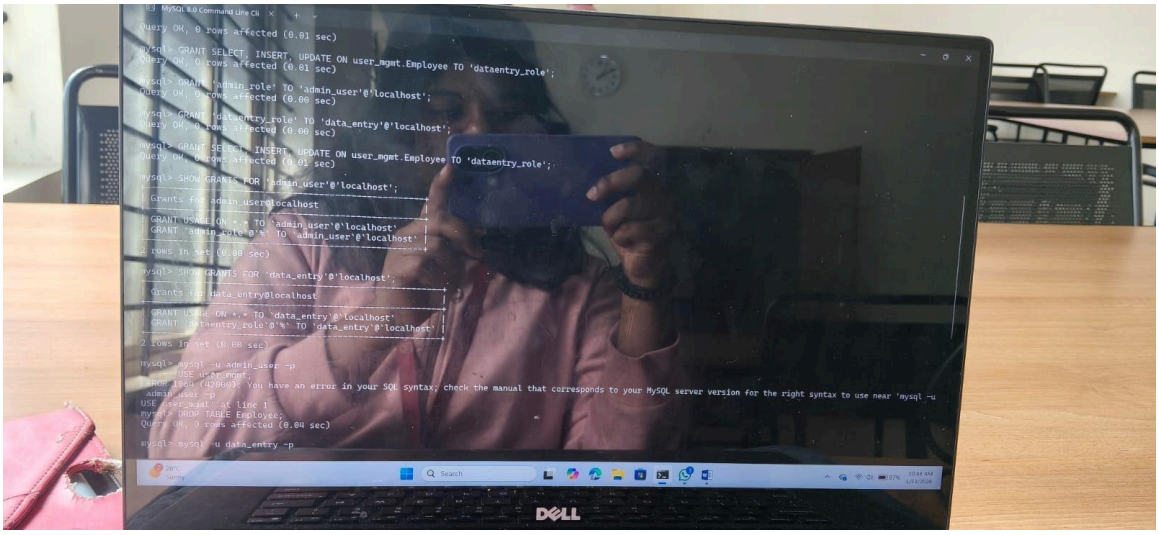

MySQL 8.0 Command Line Cli
mysql> INSERT INTO Employee (EmpName, Salary) VALUES ('Neha', 50000);
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> DELETE FROM Employee WHERE EmpID = 1;
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> REVOKE UPDATE ON user_mgmt.Employee FROM 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql> REVOKE UPDATE ON user_mgmt.Employee FROM 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)

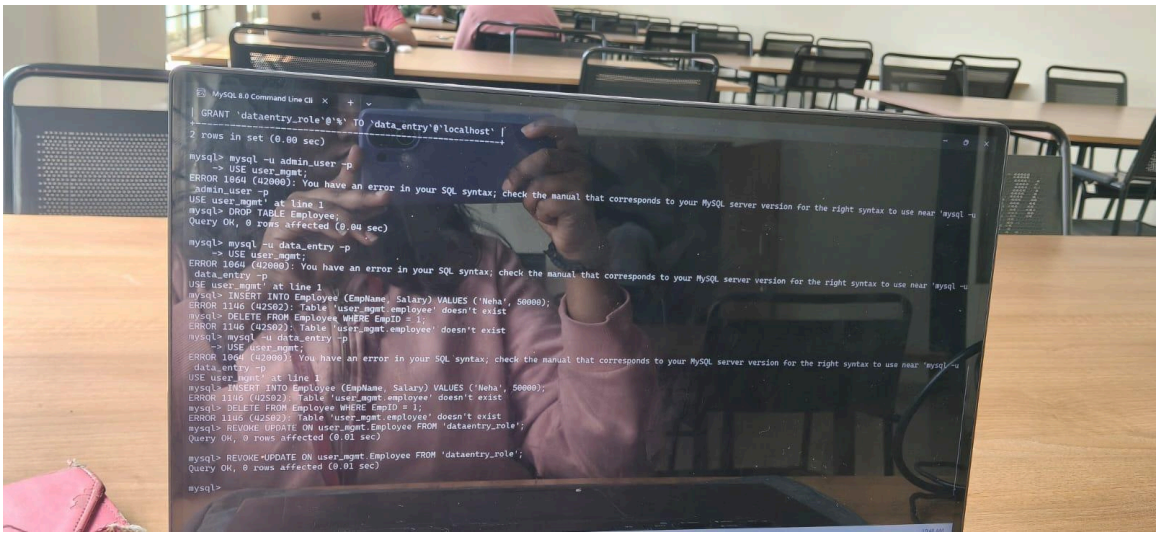

MySQL 8.0 Command Line Cli
mysql> INSERT INTO Employee (EmpName, Salary) VALUES ('Neha', 50000);
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> DELETE FROM Employee WHERE EmpID = 1;
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> REVOKE UPDATE ON user_mgmt.Employee FROM 'dataentry_role';

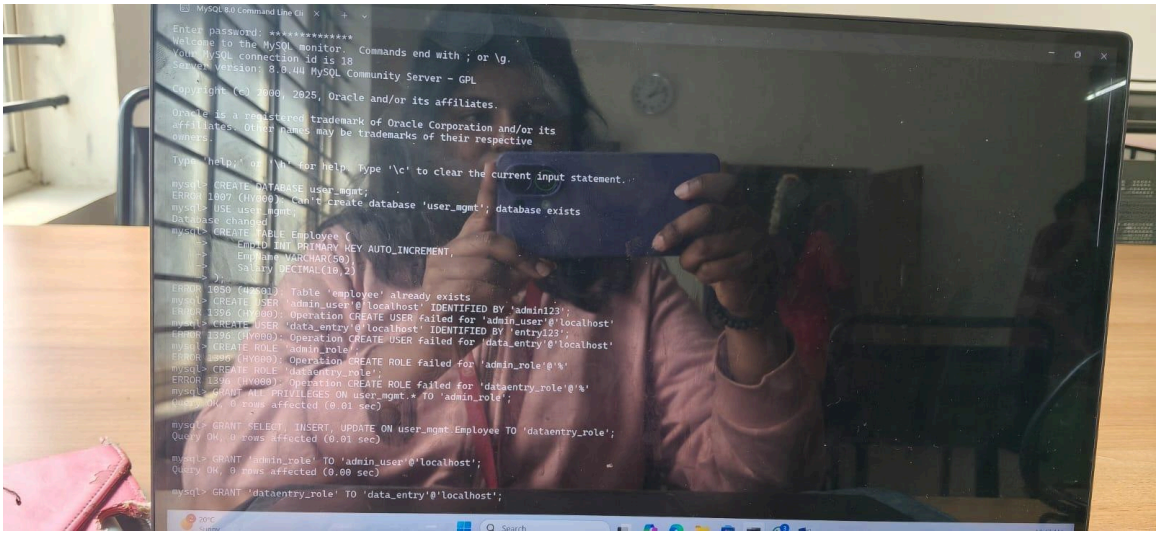

MySQL 8.0 Command Line Cli
GRANT 'dataentry_role'@'%' TO 'data_entry'@'localhost'
2 rows in set (0.00 sec)
mysql> mysql -u admin_user -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u admin_user -p
USE user_mgmt' at line 1
mysql> DROP TABLE Employee;
Query OK, 0 rows affected (0.04 sec)
mysql> mysql -u data_entry -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u data_entry -p
USE user_mgmt' at line 1
mysql> INSERT INTO Employee (EmpName, Salary) VALUES ('Neha', 50000);
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> DELETE FROM Employee WHERE EmpID = 1;
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> mysql -u data_entry -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u data_entry -p
USE user_mgmt' at line 1
mysql> INSERT INTO Employee (EmpName, Salary) VALUES ('Neha', 50000);
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> DELETE FROM Employee WHERE EmpID = 1;
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> REVOKE UPDATE ON user_mgmt.Employee FROM 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql> REVOKE UPDATE ON user_mgmt.Employee FROM 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql>

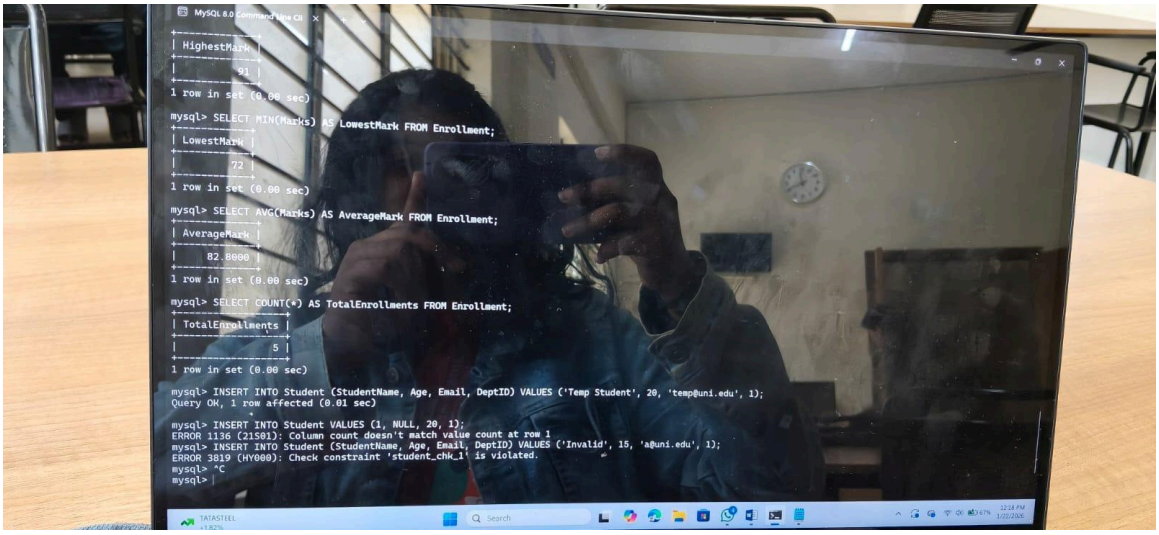

MySQL 8.0 Command Line Cli
GRANT USAGE ON *.* TO 'admin_user'@'localhost'
GRANT 'admin_role'@'%' TO 'admin_user'@'localhost'
2 rows in set (0.00 sec)
mysql> SHOW GRANTS FOR 'data_entry'@'localhost';
Grants for data_entry@localhost
GRANT USAGE ON *.* TO 'data_entry'@'localhost'
GRANT 'dataentry_role'@'%' TO 'data_entry'@'localhost'
2 rows in set (0.00 sec)
mysql> mysql -u admin_user -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u admin_user -p
USE user_mgmt' at line 1
mysql> DROP TABLE Employee;
Query OK, 0 rows affected (0.04 sec)
mysql> mysql -u data_entry -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u data_entry -p
USE user_mgmt' at line 1
mysql> INSERT INTO Employee (EmpName, Salary) VALUES ('Neha', 50000);
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> DELETE FROM Employee WHERE EmpID = 1;
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> mysql -u data_entry -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u data_entry -p
USE user_mgmt' at line 1
mysql> INSERT INTO Employee (EmpName, Salary) VALUES ('Neha', 50000);
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> DELETE FROM Employee WHERE EmpID = 1;
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> REVOKE UPDATE ON user_mgmt.Employee FROM 'dataentry_role';

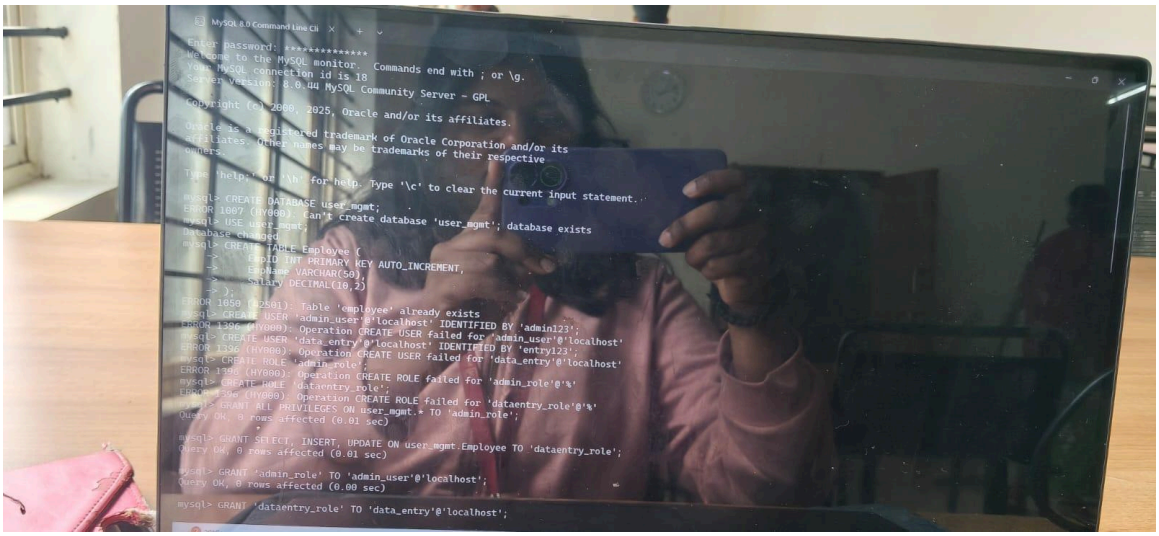

MySQL 8.0 Command Line Cli
Query OK, 0 rows affected (0.01 sec)
mysql> GRANT SELECT, INSERT, UPDATE ON user_mgmt.Employee TO 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql> GRANT 'admin_role' TO 'admin_user'@'localhost';
Query OK, 0 rows affected (0.00 sec)
mysql> GRANT 'dataentry_role' TO 'data_entry'@'localhost';
Query OK, 0 rows affected (0.00 sec)
mysql> GRANT SELECT, INSERT, UPDATE ON user_mgmt.Employee TO 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql> SHOW GRANTS FOR 'admin_user'@'localhost';
Grants for admin_user@localhost
GRANT USAGE ON *.* TO 'admin_user'@'localhost'
GRANT 'admin_role'@'%' TO 'admin_user'@'localhost'
2 rows in set (0.00 sec)
mysql> SHOW GRANTS FOR 'data_entry'@'localhost';
Grants for data_entry@localhost
GRANT USAGE ON *.* TO 'data_entry'@'localhost'
GRANT 'dataentry_role'@'%' TO 'data_entry'@'localhost'
2 rows in set (0.00 sec)
mysql> mysql -u admin_user -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u admin_user -p
USE user_mgmt' at line 1
mysql> DROP TABLE Employee;
Query OK, 0 rows affected (0.04 sec)
mysql> mysql -u data_entry -p
DELL

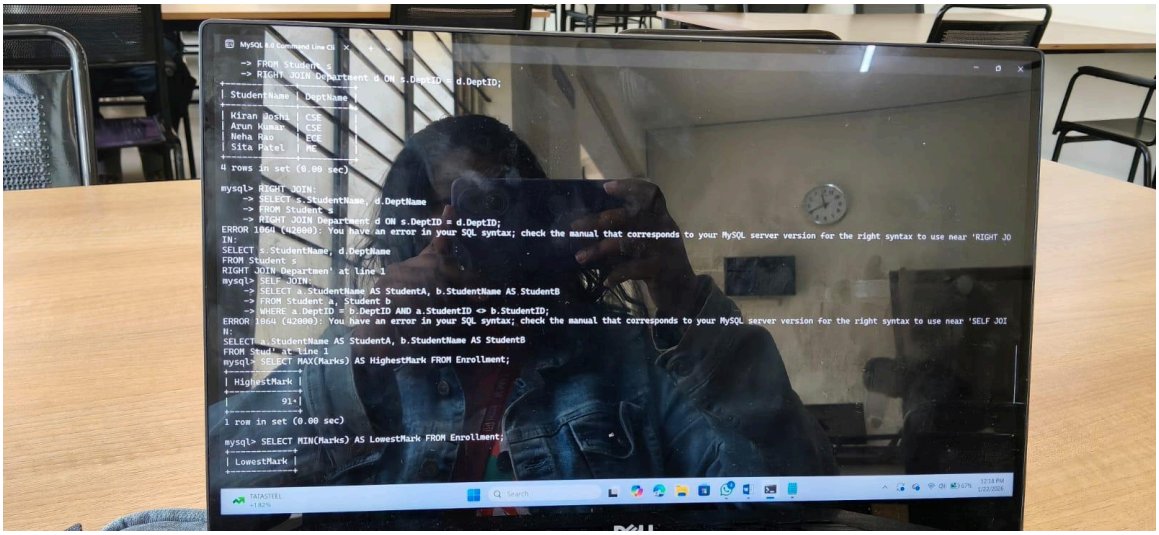

MySQL 8.0 Command Line Cli
Query OK, 0 rows affected (0.01 sec)
mysql> GRANT SELECT, INSERT, UPDATE ON user_mgmt.Employee TO 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql> GRANT 'admin_role' TO 'admin_user'@'localhost';
Query OK, 0 rows affected (0.00 sec)
mysql> GRANT 'dataentry_role' TO 'data_entry'@'localhost';
Query OK, 0 rows affected (0.00 sec)
mysql> GRANT SELECT, INSERT, UPDATE ON user_mgmt.Employee TO 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql> SHOW GRANTS FOR 'admin_user'@'localhost';
Grants for admin_user@localhost
GRANT USAGE ON *.* TO 'admin_user'@'localhost'
GRANT 'admin_role'@'%' TO 'admin_user'@'localhost'
2 rows in set (0.00 sec)
mysql> SHOW GRANTS FOR 'data_entry'@'localhost';
Grants for data_entry@localhost
GRANT USAGE ON *.* TO 'data_entry'@'localhost'
GRANT 'dataentry_role'@'%' TO 'data_entry'@'localhost'
2 rows in set (0.00 sec)
mysql> mysql -u admin_user -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u admin_user -p
USE user_mgmt' at line 1
mysql> DROP TABLE Employee;
Query OK, 0 rows affected (0.04 sec)
mysql> mysql -u data_entry -p
DELL

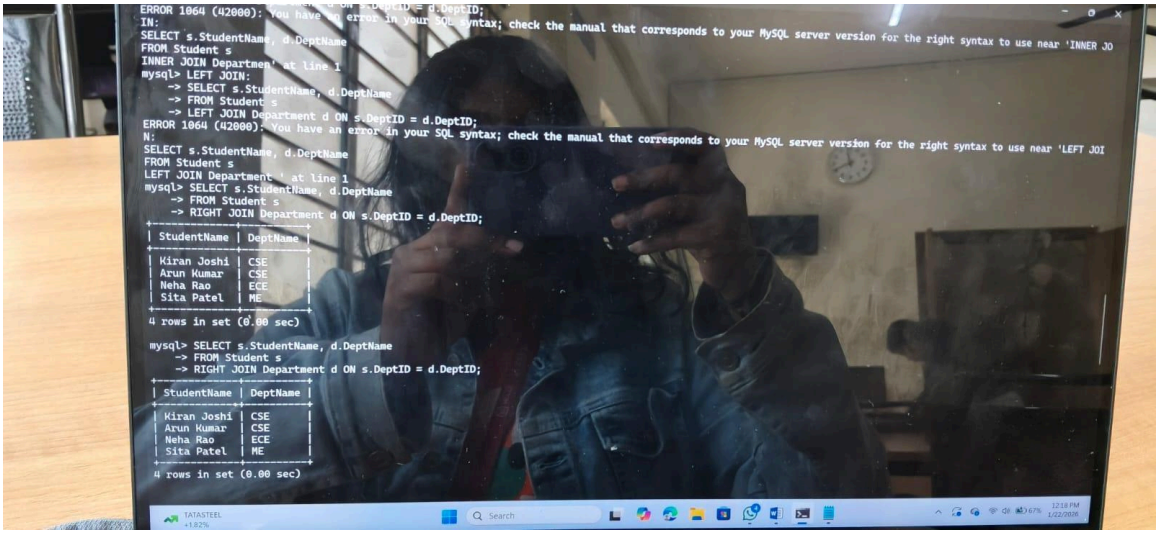

MySQL 8.0 Command Line Cli
GRANT 'dataentry_role'@'%' TO 'data_entry'@'localhost'
2 rows in set (0.00 sec)
mysql> mysql -u admin_user -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u admin_user -p
USE user_mgmt' at line 1
mysql> DROP TABLE Employee;
Query OK, 0 rows affected (0.04 sec)
mysql> mysql -u data_entry -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u data_entry -p
USE user_mgmt' at line 1
mysql> INSERT INTO Employee (EmpName, Salary) VALUES ('Neha', 50000);
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> DELETE FROM Employee WHERE EmpID = 1;
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> mysql -u data_entry -p
-> USE user_mgmt;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'mysql -u data_entry -p
USE user_mgmt' at line 1
mysql> INSERT INTO Employee (EmpName, Salary) VALUES ('Neha', 50000);
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> DELETE FROM Employee WHERE EmpID = 1;
ERROR 1146 (42S02): Table 'user_mgmt.employee' doesn't exist
mysql> REVOKE UPDATE ON user_mgmt.Employee FROM 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql> REVOKE UPDATE ON user_mgmt.Employee FROM 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql>

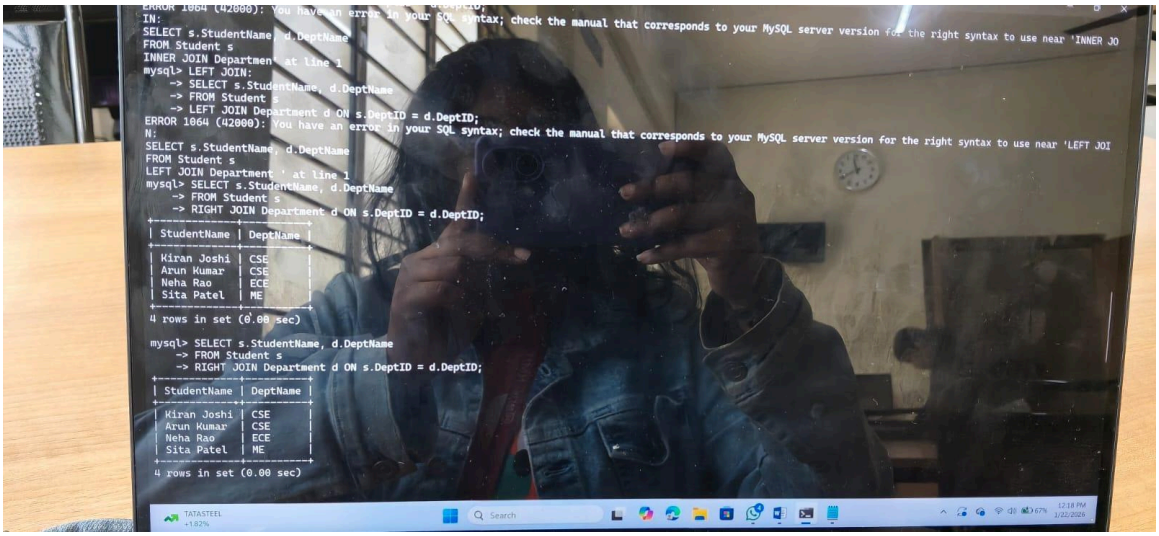

MySQL 8.0 Command Line Cli
Enter password: **********
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 18
Server version: 8.0.44 MySQL Community Server - GPL
Copyright (c) 2000, 2025, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE DATABASE user_mgmt;
ERROR 1007 (HY000): Can't create database 'user_mgmt'; database exists
mysql> USE user_mgmt;
Database changed
mysql> CREATE TABLE Employee (
-> EmpID INT PRIMARY KEY AUTO_INCREMENT,
-> EmpName VARCHAR(50),
-> Salary DECIMAL(10,2)
->);
ERROR 1050 (42S01): Table 'employee' already exists
mysql> CREATE USER 'admin_user'@'localhost' IDENTIFIED BY 'admin123';
ERROR 1396 (HY000): Operation CREATE USER failed for 'admin_user'@'localhost'
mysql> CREATE USER 'data_entry'@'localhost' IDENTIFIED BY 'entry123';
ERROR 1396 (HY000): Operation CREATE USER failed for 'data_entry'@'localhost'
mysql> CREATE ROLE 'admin_role';
ERROR 1396 (HY000): Operation CREATE ROLE failed for 'admin_role'@'%'
mysql> CREATE ROLE 'dataentry_role';
ERROR 1396 (HY000): Operation CREATE ROLE failed for 'dataentry_role'@'%'
mysql> GRANT ALL PRIVILEGES ON user_mgmt.* TO 'admin_role';
Query OK, 0 rows affected (0.01 sec)
mysql> GRANT SELECT, INSERT, UPDATE ON user_mgmt.Employee TO 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql> GRANT 'admin_role' TO 'admin_user'@'localhost';
Query OK, 0 rows affected (0.00 sec)
mysql> GRANT 'dataentry_role' TO 'data_entry'@'localhost';

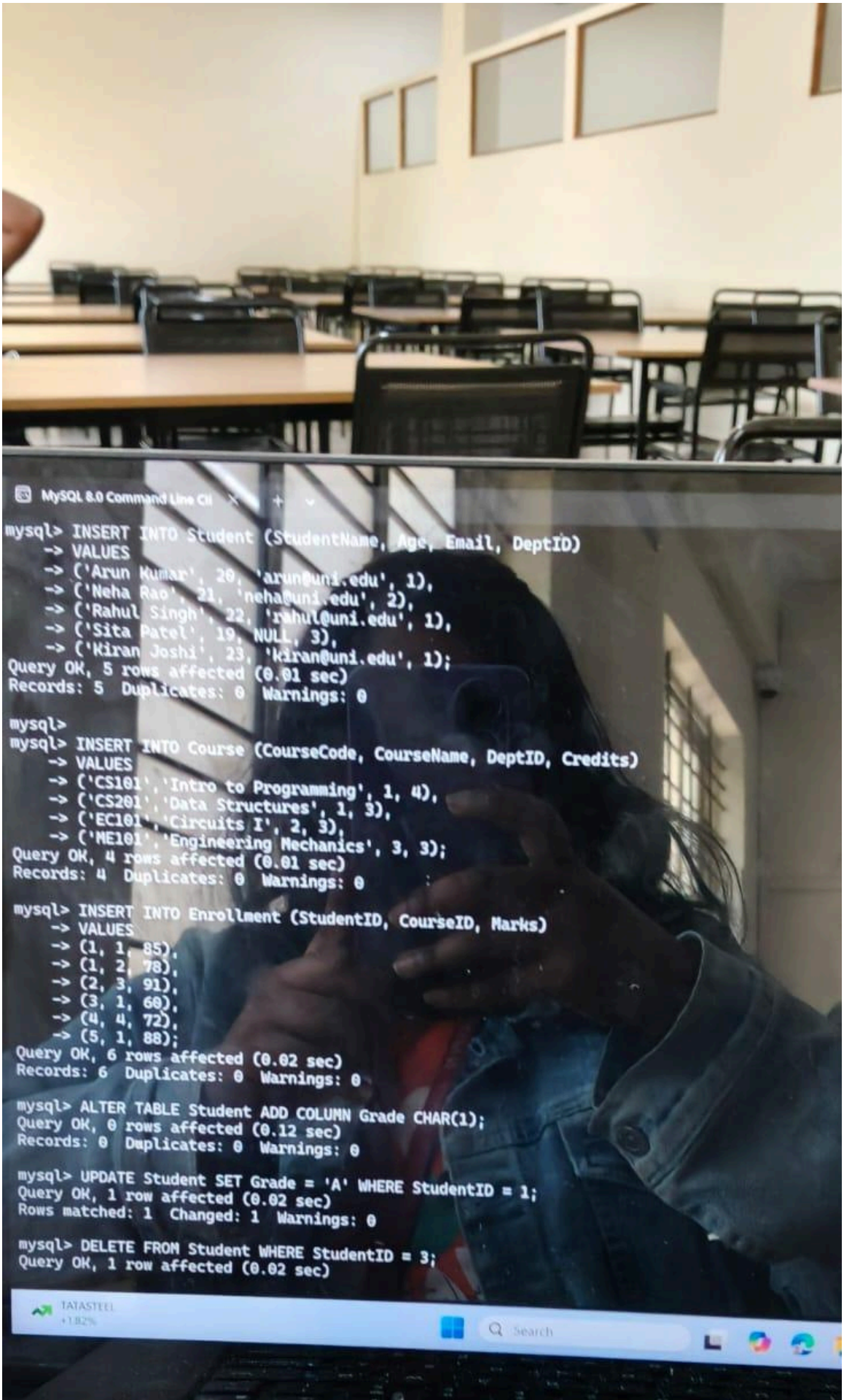

MySQL 8.0 Command Line Cli
| HighestMark |
| 91 |
1 row in set (0.00 sec)
mysql> SELECT MIN(Marks) AS LowestMark FROM Enrollment;
| LowestMark |
| 72 |
1 row in set (0.00 sec)
mysql> SELECT AVG(Marks) AS AverageMark FROM Enrollment;
| AverageMark |
| 82.8000 |
1 row in set (0.00 sec)
mysql> SELECT COUNT(*) AS TotalEnrollments FROM Enrollment;
| TotalEnrollments |
| 5 |
1 row in set (0.00 sec)
mysql> INSERT INTO Student (StudentName, Age, Email, DeptID) VALUES ('Temp Student', 20, 'temp@uni.edu', 1);
Query OK, 1 row affected (0.01 sec)
mysql> INSERT INTO Student VALUES (1, NULL, 20, 1);
ERROR 1136 (21S01): Column count doesn't match value count at row 1
mysql> INSERT INTO Student (StudentName, Age, Email, DeptID) VALUES ('Invalid', 15, 'a@uni.edu', 1);
ERROR 3819 (HY000): Check constraint 'student_chk_1' is violated.
mysql> ^C
mysql>

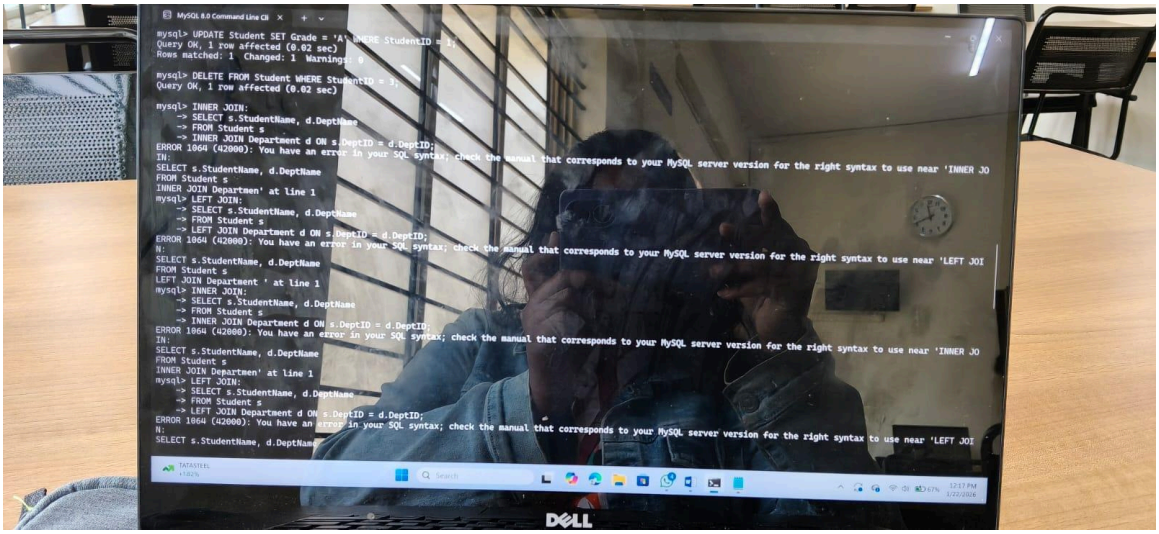

MySQL 8.0 Command Line Cli
Enter password: **********
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 18
Server version: 8.0.44 MySQL Community Server - GPL
Copyright (c) 2000, 2025, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE DATABASE user_mgmt;
ERROR 1007 (HY000): Can't create database 'user_mgmt'; database exists
mysql> USE user_mgmt;
Database changed
mysql> CREATE TABLE Employee (
-> EmpID INT PRIMARY KEY AUTO_INCREMENT,
-> EmpName VARCHAR(50),
-> Salary DECIMAL(10,2)
->);
ERROR 1050 (42S01): Table 'employee' already exists
mysql> CREATE USER 'admin_user'@'localhost' IDENTIFIED BY 'admin123';
ERROR 1396 (HY000): Operation CREATE USER failed for 'admin_user'@'localhost'
mysql> CREATE USER 'data_entry'@'localhost' IDENTIFIED BY 'entry123';
ERROR 1396 (HY000): Operation CREATE USER failed for 'data_entry'@'localhost'
mysql> CREATE ROLE 'admin_role';
ERROR 1396 (HY000): Operation CREATE ROLE failed for 'admin_role'@'%'
mysql> CREATE ROLE 'dataentry_role';
ERROR 1396 (HY000): Operation CREATE ROLE failed for 'dataentry_role'@'%'
mysql> GRANT ALL PRIVILEGES ON user_mgmt.* TO 'admin_role';
Query OK, 0 rows affected (0.01 sec)
mysql> GRANT SELECT, INSERT, UPDATE ON user_mgmt.Employee TO 'dataentry_role';
Query OK, 0 rows affected (0.01 sec)
mysql> GRANT 'admin_role' TO 'admin_user'@'localhost';
Query OK, 0 rows affected (0.00 sec)
mysql> GRANT 'dataentry_role' TO 'data_entry'@'localhost';

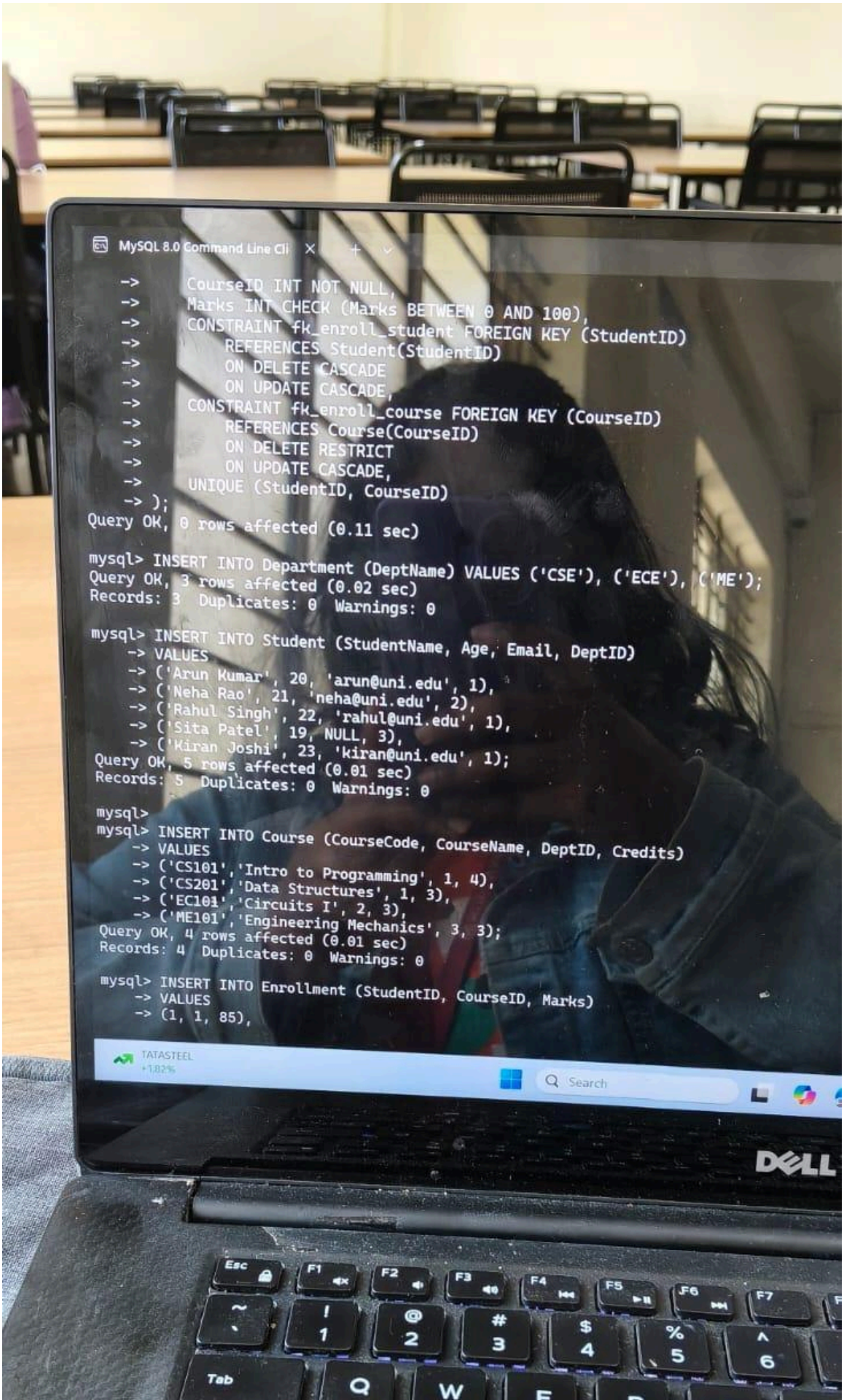

MySQL 8.0 Command Line Cli
-> FROM Student s
-> RIGHT JOIN Department d ON s.DeptID = d.DeptID;
StudentName	DeptName
Kiran Joshi	CSE
Arun Kumar	CSE
Neha Rao	ECE
Sita Patel	ME
4 rows in set (0.00 sec)
mysql> RIGHT JOIN:
-> SELECT s.StudentName, d.DeptName
-> FROM Student s
-> RIGHT JOIN Department d ON s.DeptID = d.DeptID;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'RIGHT JOIN:
SELECT s.StudentName, d.DeptName
FROM Student s
RIGHT JOIN Department' at line 1
mysql> SELF JOIN:
-> SELECT a.StudentName AS StudentA, b.StudentName AS StudentB
-> FROM Student a, Student b
-> WHERE a.DeptID = b.DeptID AND a.StudentID <> b.StudentID;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'SELF JOIN:
SELECT a.StudentName AS StudentA, b.StudentName AS StudentB
FROM Student' at line 1
mysql> SELECT MAX(Marks) AS HighestMark FROM Enrollment;
| HighestMark |
| 91 |
1 row in set (0.00 sec)
mysql> SELECT MIN(Marks) AS LowestMark FROM Enrollment;
| LowestMark |

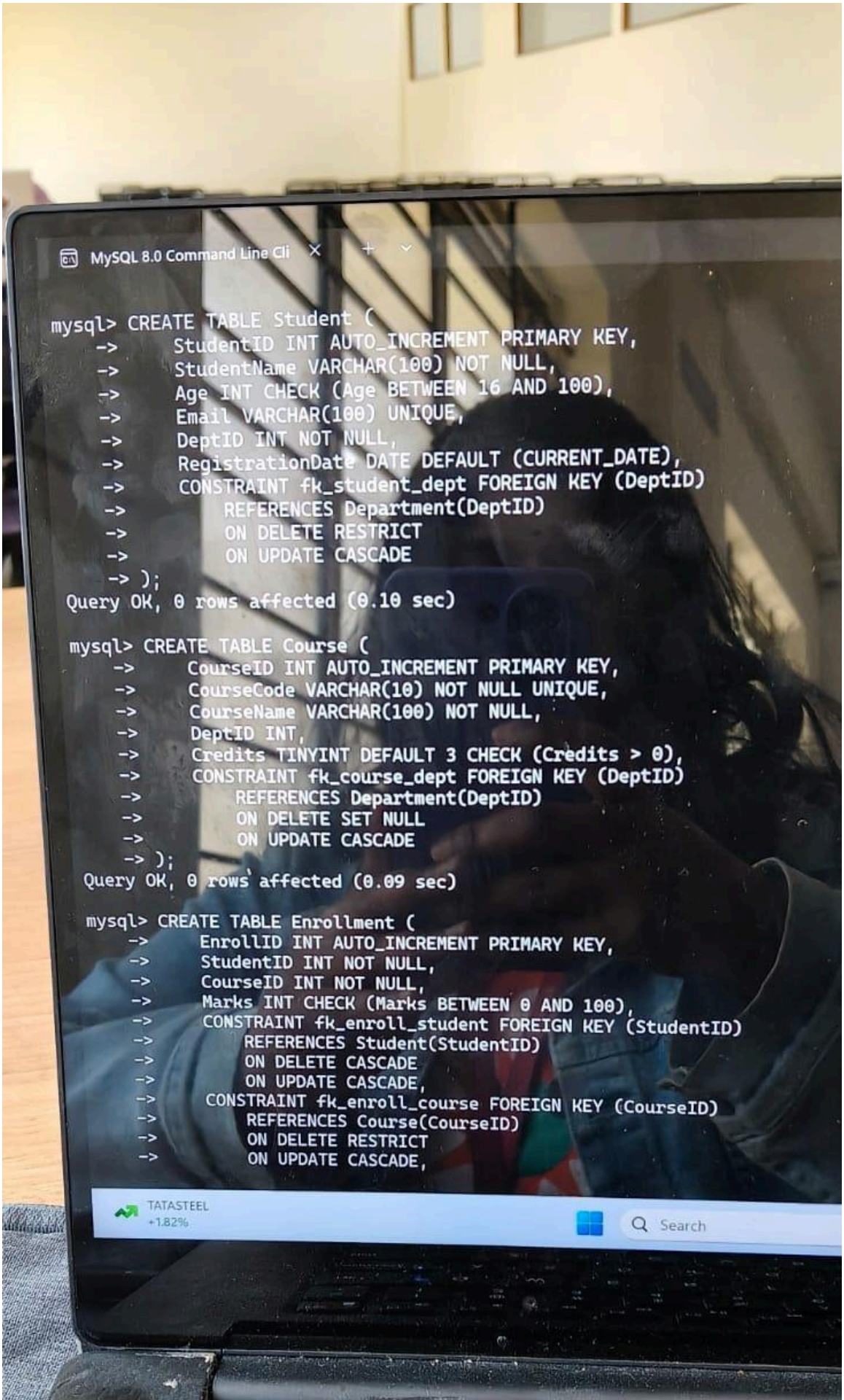

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'INNER JO
IN:
SELECT s.StudentName, d.DeptName
FROM Student s
INNER JOIN Departmen' at line 1
mysql> LEFT JOIN:
-> SELECT s.StudentName, d.DeptName
-> FROM Student s
-> LEFT JOIN Department d ON s.DeptID = d.DeptID;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'LEFT JOI
N:
SELECT s.StudentName, d.DeptName
FROM Student s
LEFT JOIN Department ' at line 1
mysql> SELECT s.StudentName, d.DeptName
-> FROM Student s
-> RIGHT JOIN Department d ON s.DeptID = d.DeptID;
StudentName	DeptName
Kiran Joshi	CSE
Arun Kumar	CSE
Neha Rao	ECE
Sita Patel	ME
4 rows in set (0.00 sec)
mysql> SELECT s.StudentName, d.DeptName
-> FROM Student s
-> RIGHT JOIN Department d ON s.DeptID = d.DeptID;
StudentName	DeptName
Kiran Joshi	CSE
Arun Kumar	CSE
Neha Rao	ECE
Sita Patel	ME
4 rows in set (0.00 sec)

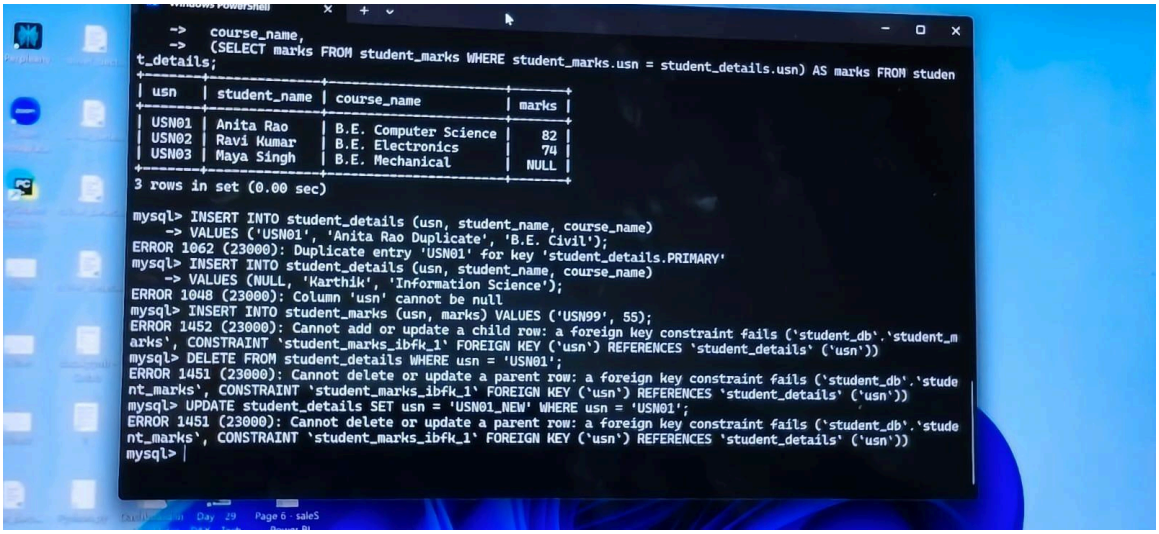

IN:
SELECT s.StudentName, d.DeptName
FROM Student s
INNER JOIN Departmen' at line 1
mysql> LEFT JOIN:
-> SELECT s.StudentName, d.DeptName
-> FROM Student s
-> LEFT JOIN Department d ON s.DeptID = d.DeptID;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'LEFT JOI
N:
SELECT s.StudentName, d.DeptName
FROM Student s
LEFT JOIN Department ' at line 1
mysql> SELECT s.StudentName, d.DeptName
-> FROM Student s
-> RIGHT JOIN Department d ON s.DeptID = d.DeptID;
StudentName	DeptName
Kiran Joshi	CSE
Arun Kumar	CSE
Neha Rao	ECE
Sita Patel	ME
4 rows in set (0.00 sec)
mysql> SELECT s.StudentName, d.DeptName
-> FROM Student s
-> RIGHT JOIN Department d ON s.DeptID = d.DeptID;
StudentName	DeptName
Kiran Joshi	CSE
Arun Kumar	CSE
Neha Rao	ECE
Sita Patel	ME
4 rows in set (0.00 sec)

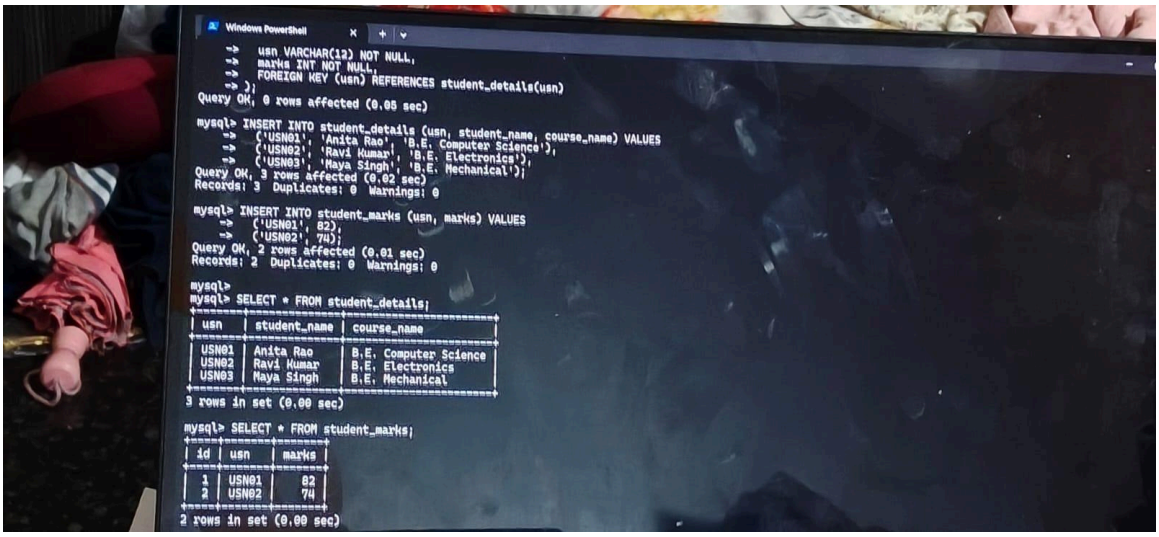

MySQL 8.0 Command Line Cli
mysql> INSERT INTO Student (StudentName, Age, Email, DeptID)
-> VALUES
-> ('Arun Kumar', 20, 'arun@uni.edu', 1),
-> ('Neha Rao', 21, 'neha@uni.edu', 2),
-> ('Rahul Singh', 22, 'rahul@uni.edu', 1),
-> ('Sita Patel', 19, NULL, 3),
-> ('Kiran Joshi', 23, 'kiran@uni.edu', 1);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO Course (CourseCode, CourseName, DeptID, Credits)
-> VALUES
-> ('CS101','Intro to Programming', 1, 4),
-> ('CS201','Data Structures', 1, 3),
-> ('EC101','Circuits I', 2, 3),
-> ('ME101','Engineering Mechanics', 3, 3);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
mysql> INSERT INTO Enrollment (StudentID, CourseID, Marks)
-> VALUES
-> (1, 1, 85),
-> (1, 2, 78),
-> (2, 3, 91),
-> (3, 1, 60),
-> (4, 4, 72),
-> (5, 1, 88);
Query OK, 6 rows affected (0.02 sec)
Records: 6 Duplicates: 0 Warnings: 0
mysql> ALTER TABLE Student ADD COLUMN Grade CHAR(1);
Query OK, 0 rows affected (0.12 sec)
Records: 0 Duplicates: 0 Warnings: 0
mysql> UPDATE Student SET Grade = 'A' WHERE StudentID = 1;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1 Changed: 1 Warnings: 0
mysql> DELETE FROM Student WHERE StudentID = 3;
Query OK, 1 row affected (0.02 sec)
TATASTEEL
+1.82%
Search

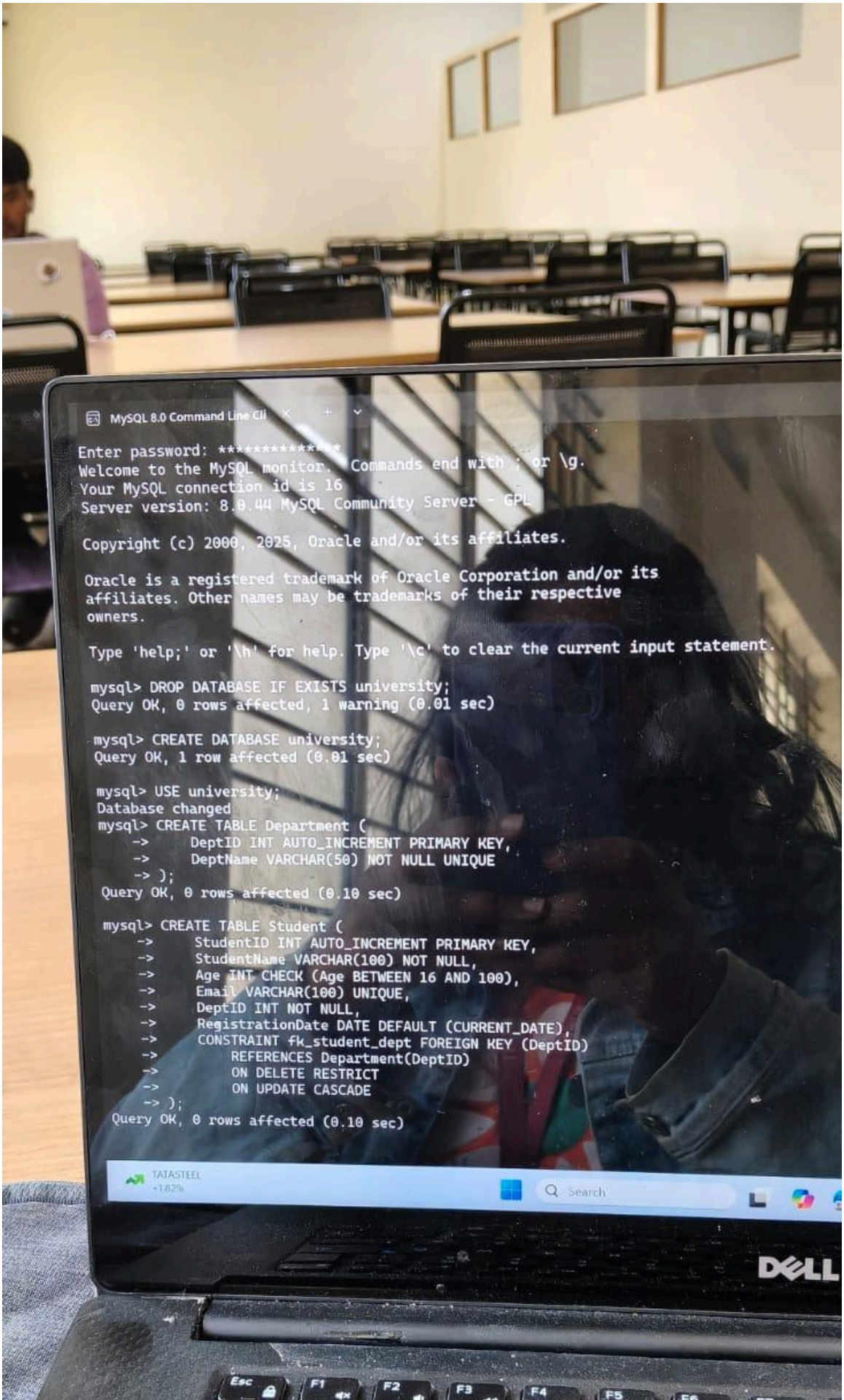

MySQL 8.0 Command Line Cli
mysql> UPDATE Student SET Grade = 'A' WHERE StudentID = 1;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1 Changed: 1 Warnings: 0
mysql> DELETE FROM Student WHERE StudentID = 3;
Query OK, 1 row affected (0.02 sec)
mysql> INNER JOIN:
-> SELECT s.StudentName, d.DeptName
-> FROM Student s
-> INNER JOIN Department d ON s.DeptID = d.DeptID;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'INNER JO
IN:
SELECT s.StudentName, d.DeptName
FROM Student s
INNER JOIN Departmen' at line 1
mysql> LEFT JOIN:
-> SELECT s.StudentName, d.DeptName
-> FROM Student s
-> LEFT JOIN Department d ON s.DeptID = d.DeptID;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'LEFT JOI
N:
SELECT s.StudentName, d.DeptName
FROM Student s
LEFT JOIN Department ' at line 1
mysql> INNER JOIN:
-> SELECT s.StudentName, d.DeptName
-> FROM Student s
-> INNER JOIN Department d ON s.DeptID = d.DeptID;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'INNER JO
IN:
SELECT s.StudentName, d.DeptName
FROM Student s
INNER JOIN Departmen' at line 1
mysql> LEFT JOIN:
-> SELECT s.StudentName, d.DeptName
-> FROM Student s
-> LEFT JOIN Department d ON s.DeptID = d.DeptID;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'LEFT JOI
N:
SELECT s.StudentName, d.DeptName
DELL

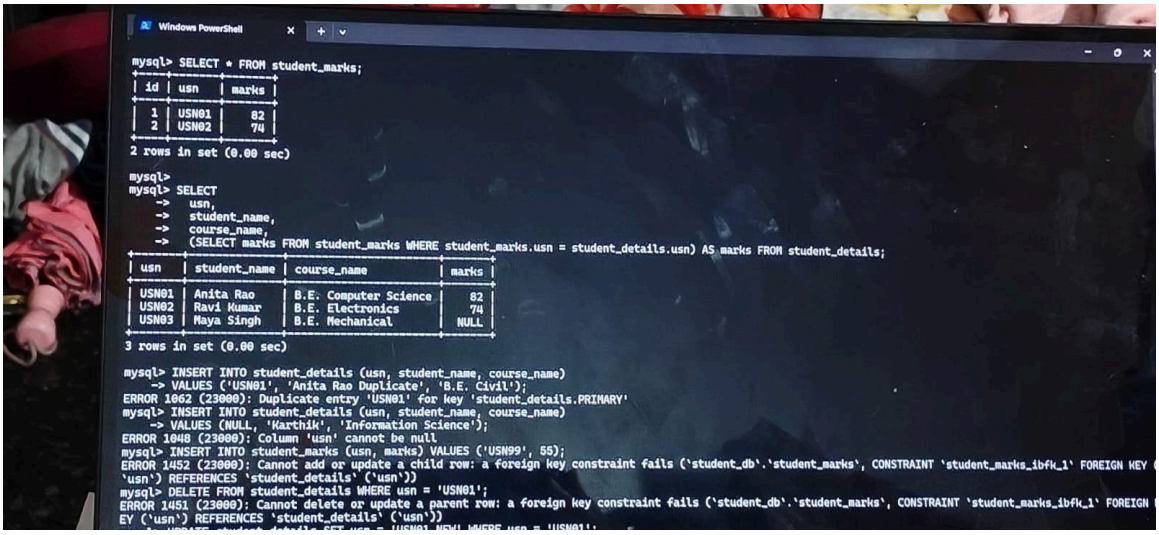

MySQL 8.0 Command Line Cli
-> CourseID INT NOT NULL,
-> Marks INT CHECK (Marks BETWEEN 0 AND 100),
-> CONSTRAINT fk_enroll_student FOREIGN KEY (StudentID)
-> REFERENCES Student(StudentID)
-> ON DELETE CASCADE
-> ON UPDATE CASCADE,
-> CONSTRAINT fk_enroll_course FOREIGN KEY (CourseID)
-> REFERENCES Course(CourseID)
-> ON DELETE RESTRICT
-> ON UPDATE CASCADE,
-> UNIQUE (StudentID, CourseID)
->);
Query OK, 0 rows affected (0.11 sec)
mysql> INSERT INTO Department (DeptName) VALUES ('CSE'), ('ECE'), ('ME');
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0
mysql> INSERT INTO Student (StudentName, Age, Email, DeptID)
-> VALUES
-> ('Arun Kumar', 20, 'arun@uni.edu', 1),
-> ('Neha Rao', 21, 'neha@uni.edu', 2),
-> ('Rahul Singh', 22, 'rahul@uni.edu', 1),
-> ('Sita Patel', 19, NULL, 3),
-> ('Kiran Joshi', 23, 'kiran@uni.edu', 1);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO Course (CourseCode, CourseName, DeptID, Credits)
-> VALUES
-> ('CS101','Intro to Programming', 1, 4),
-> ('CS201','Data Structures', 1, 3),
-> ('EC101','Circuits I', 2, 3),
-> ('ME101','Engineering Mechanics', 3, 3);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
mysql> INSERT INTO Enrollment (StudentID, CourseID, Marks)
-> VALUES
-> (1, 1, 85),
TATASTEEL
+1.02%
Search
DELL

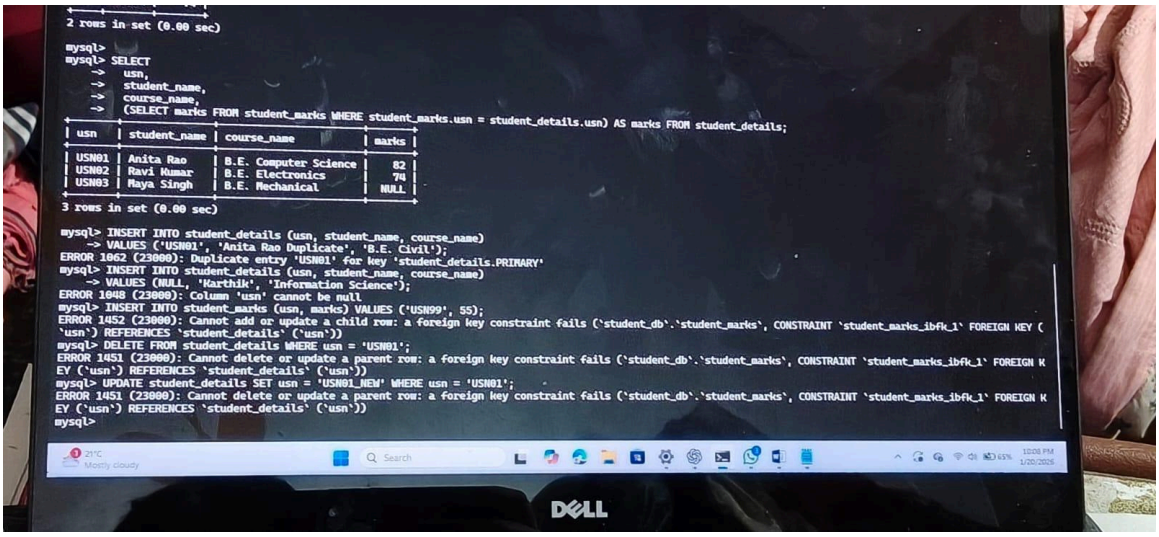

MySQL 8.0 Command Line Cli
mysql> CREATE TABLE Student (
-> StudentID INT AUTO_INCREMENT PRIMARY KEY,
-> StudentName VARCHAR(100) NOT NULL,
-> Age INT CHECK (Age BETWEEN 16 AND 100),
-> Email VARCHAR(100) UNIQUE,
-> DeptID INT NOT NULL,
-> RegistrationDate DATE DEFAULT (CURRENT_DATE),
-> CONSTRAINT fk_student_dept FOREIGN KEY (DeptID)
-> REFERENCES Department(DeptID)
-> ON DELETE RESTRICT
-> ON UPDATE CASCADE
->);
Query OK, 0 rows affected (0.10 sec)
mysql> CREATE TABLE Course (
-> CourseID INT AUTO_INCREMENT PRIMARY KEY,
-> CourseCode VARCHAR(10) NOT NULL UNIQUE,
-> CourseName VARCHAR(100) NOT NULL,
-> DeptID INT,
-> Credits TINYINT DEFAULT 3 CHECK (Credits > 0),
-> CONSTRAINT fk_course_dept FOREIGN KEY (DeptID)
-> REFERENCES Department(DeptID)
-> ON DELETE SET NULL
-> ON UPDATE CASCADE
->);
Query OK, 0 rows affected (0.09 sec)
mysql> CREATE TABLE Enrollment (
-> EnrollID INT AUTO_INCREMENT PRIMARY KEY,
-> StudentID INT NOT NULL,
-> CourseID INT NOT NULL,
-> Marks INT CHECK (Marks BETWEEN 0 AND 100),
-> CONSTRAINT fk_enroll_student FOREIGN KEY (StudentID)
-> REFERENCES Student(StudentID)
-> ON DELETE CASCADE
-> ON UPDATE CASCADE,
-> CONSTRAINT fk_enroll_course FOREIGN KEY (CourseID)
-> REFERENCES Course(CourseID)
-> ON DELETE RESTRICT
-> ON UPDATE CASCADE,
TATASTEEL
+1.82%
Search

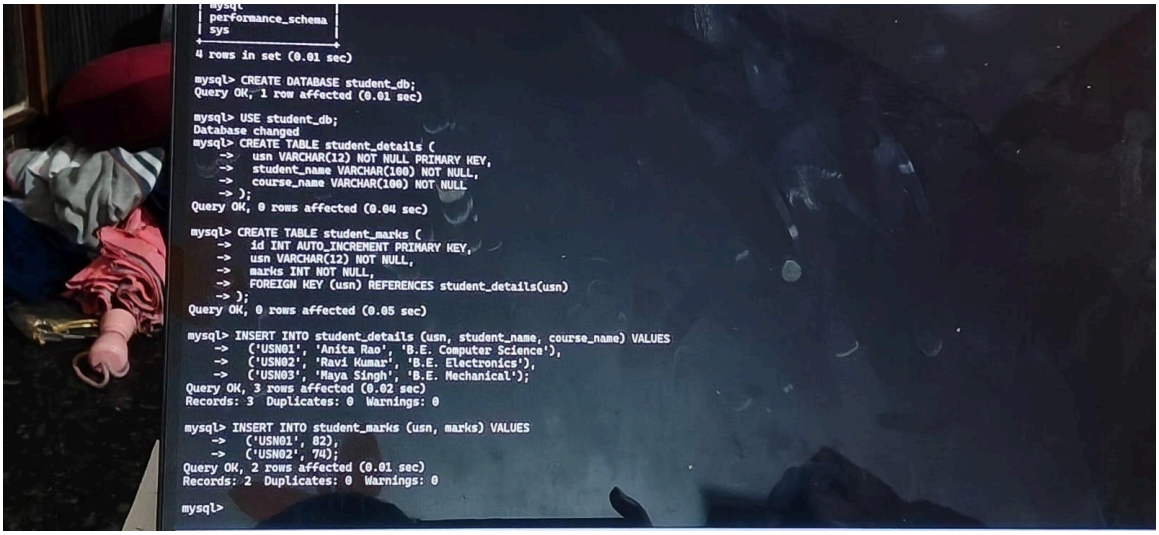

-> course_name,
-> (SELECT marks FROM student_marks WHERE student_marks.usn = student_details.usn) AS marks FROM studen
t_details;
usn	student_name	course_name	marks
USN01	Anita Rao	B.E. Computer Science	82
USN02	Ravi Kumar	B.E. Electronics	74
USN03	Maya Singh	B.E. Mechanical	NULL
3 rows in set (0.00 sec)
mysql> INSERT INTO student_details (usn, student_name, course_name)
-> VALUES ('USN01', 'Anita Rao Duplicate', 'B.E. Civil');
ERROR 1062 (23000): Duplicate entry 'USN01' for key 'student_details.PRIMARY'
mysql> INSERT INTO student_details (usn, student_name, course_name)
-> VALUES (NULL, 'Karthik', 'Information Science');
ERROR 1048 (23000): Column 'usn' cannot be null
mysql> INSERT INTO student_marks (usn, marks) VALUES ('USN99', 55);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`student_db`.`student_m
arks`, CONSTRAINT `student_marks_ibfk_1` FOREIGN KEY (`usn`) REFERENCES `student_details` (`usn`))
mysql> DELETE FROM student_details WHERE usn = 'USN01';
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`student_db`.`stude
nt_marks`, CONSTRAINT `student_marks_ibfk_1` FOREIGN KEY (`usn`) REFERENCES `student_details` (`usn`))
mysql> UPDATE student_details SET usn = 'USN01_NEW' WHERE usn = 'USN01';
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`student_db`.`stude
nt_marks`, CONSTRAINT `student_marks_ibfk_1` FOREIGN KEY (`usn`) REFERENCES `student_details` (`usn`))
mysql>

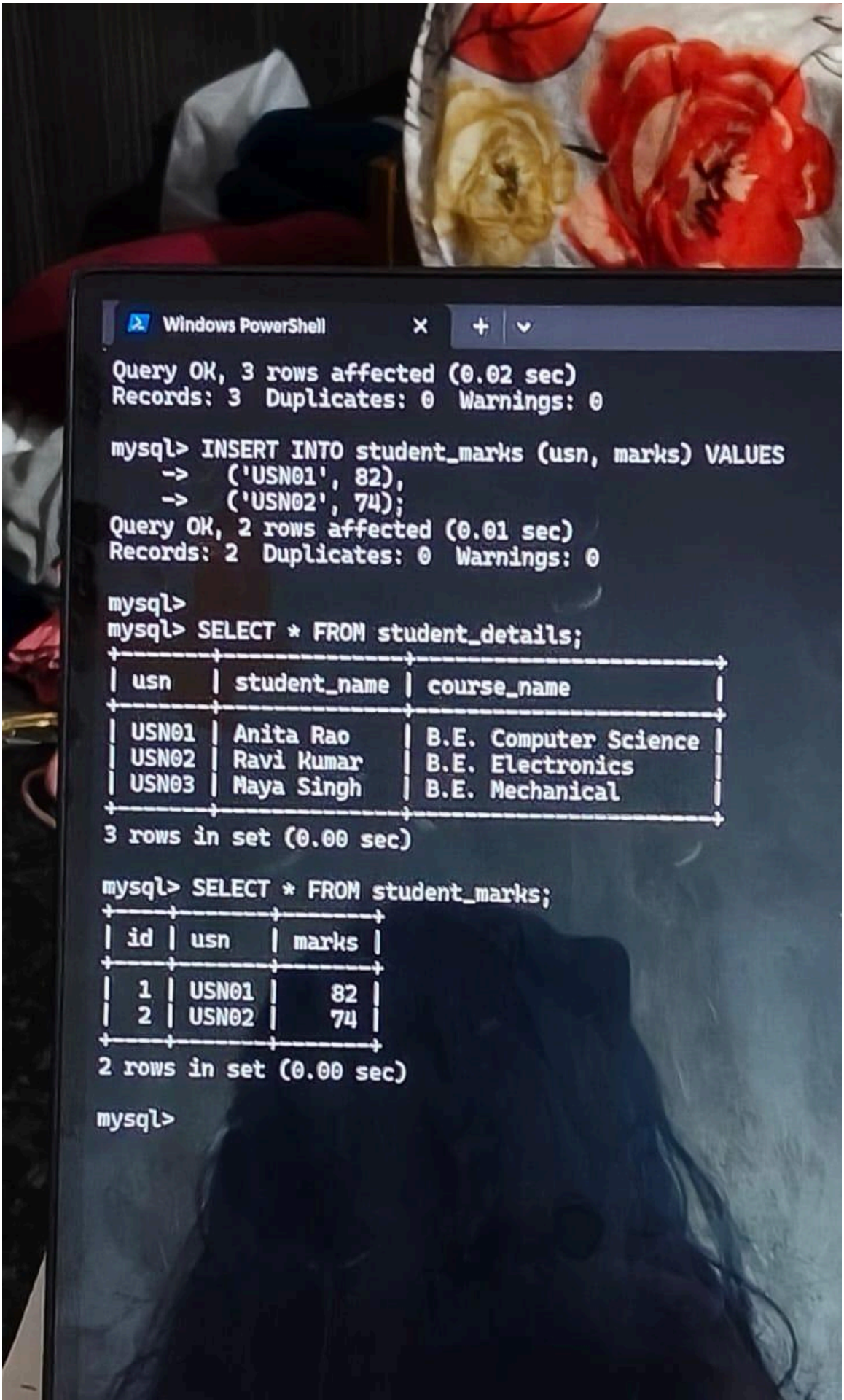

Windows PowerShell
-> usn VARCHAR(12) NOT NULL,
-> marks INT NOT NULL,
-> FOREIGN KEY (usn) REFERENCES student_details(usn)
->);
Query OK, 0 rows affected (0.05 sec)
mysql> INSERT INTO student_details (usn, student_name, course_name) VALUES
-> ('USN01', 'Anita Rao', 'B.E. Computer Science'),
-> ('USN02', 'Ravi Kumar', 'B.E. Electronics'),
-> ('USN03', 'Maya Singh', 'B.E. Mechanical');
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0
mysql> INSERT INTO student_marks (usn, marks) VALUES
-> ('USN01', 82),
-> ('USN02', 74);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> SELECT * FROM student_details;
usn	student_name	course_name
USN01	Anita Rao	B.E. Computer Science
USN02	Ravi Kumar	B.E. Electronics
USN03	Maya Singh	B.E. Mechanical
3 rows in set (0.00 sec)
mysql> SELECT * FROM student_marks;
id	usn	marks
1	USN01	82
2	USN02	74
2 rows in set (0.00 sec)

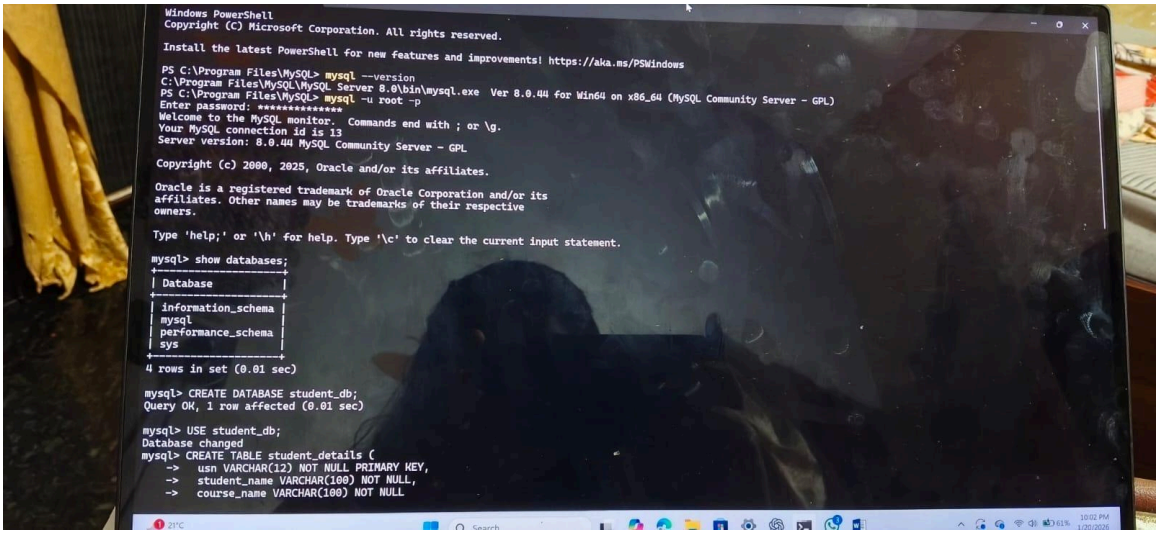

MySQL 8.0 Command Line Cli
Enter password: **********
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 16
Server version: 8.0.44 MySQL Community Server - GPL
Copyright (c) 2000, 2025, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> DROP DATABASE IF EXISTS university;
Query OK, 0 rows affected, 1 warning (0.01 sec)
mysql> CREATE DATABASE university;
Query OK, 1 row affected (0.01 sec)
mysql> USE university;
Database changed
mysql> CREATE TABLE Department (
-> DeptID INT AUTO_INCREMENT PRIMARY KEY,
-> DeptName VARCHAR(50) NOT NULL UNIQUE
->);
Query OK, 0 rows affected (0.10 sec)
mysql> CREATE TABLE Student (
-> StudentID INT AUTO_INCREMENT PRIMARY KEY,
-> StudentName VARCHAR(100) NOT NULL,
-> Age INT CHECK (Age BETWEEN 16 AND 100),
-> Email VARCHAR(100) UNIQUE,
-> DeptID INT NOT NULL,
-> RegistrationDate DATE DEFAULT (CURRENT_DATE),
-> CONSTRAINT fk_student_dept FOREIGN KEY (DeptID)
-> REFERENCES Department(DeptID)
-> ON DELETE RESTRICT
-> ON UPDATE CASCADE
->);
Query OK, 0 rows affected (0.10 sec)
TATASTEEL
+1.82%
Search
DELL

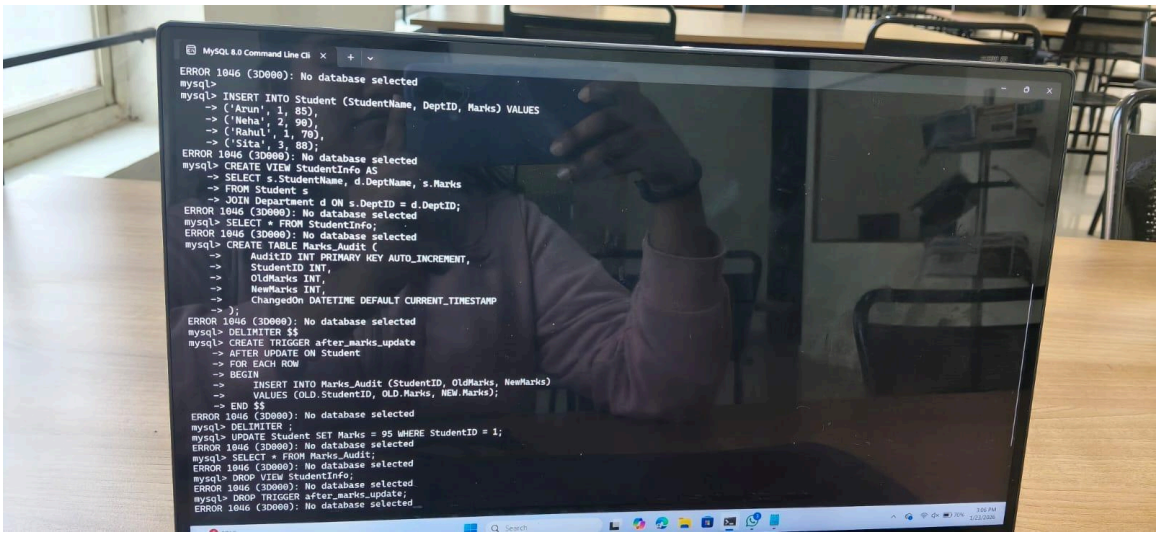

Windows PowerShell
mysql> SELECT * FROM student_marks;
id	usn	marks
1	USN01	82
2	USN02	74
2 rows in set (0.00 sec)
mysql> SELECT
-> usn,
-> student_name,
-> course_name,
-> (SELECT marks FROM student_marks WHERE student_marks.usn = student_details.usn) AS marks FROM student_details;
usn	student_name	course_name	marks
USN01	Anita Rao	B.E. Computer Science	82
USN02	Ravi Kumar	B.E. Electronics	74
USN03	Maya Singh	B.E. Mechanical	NULL
3 rows in set (0.00 sec)
mysql> INSERT INTO student_details (usn, student_name, course_name)
-> VALUES ('USN01', 'Anita Rao Duplicate', 'B.E. Civil');
ERROR 1062 (23000): Duplicate entry 'USN01' for key 'student_details.PRIMARY'
mysql> INSERT INTO student_details (usn, student_name, course_name)
-> VALUES (NULL, 'Karthik', 'Information Science');
ERROR 1048 (23000): Column 'usn' cannot be null
mysql> INSERT INTO student_marks (usn, marks) VALUES ('USN99', 55);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`student_db`.`student_marks`, CONSTRAINT `student_marks_ibfk_1` FOREIGN KEY
`usn`) REFERENCES `student_details` (`usn`))
mysql> DELETE FROM student_details WHERE usn = 'USN01';
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`student_db`.`student_marks`, CONSTRAINT `student_marks_ibfk_1` FOREIGN
EY (`usn`) REFERENCES `student_details` (`usn`))


2 rows in set (0.00 sec)
mysql>
mysql> SELECT
-> usn,
-> student_name,
-> course_name,
-> (SELECT marks FROM student_marks WHERE student_marks.usn = student_details.usn) AS marks FROM student_details;
usn	student_name	course_name	marks
USN01	Anita Rao	B.E. Computer Science	82
USN02	Ravi Kumar	B.E. Electronics	74
USN03	Maya Singh	B.E. Mechanical	NULL
3 rows in set (0.00 sec)
mysql> INSERT INTO student_details (usn, student_name, course_name)
-> VALUES ('USN01', 'Anita Rao Duplicate', 'B.E. Civil');
ERROR 1062 (23000): Duplicate entry 'USN01' for key 'student_details.PRIMARY'
mysql> INSERT INTO student_details (usn, student_name, course_name)
-> VALUES (NULL, 'Karthik', 'Information Science');
ERROR 1048 (23000): Column 'usn' cannot be null
mysql> INSERT INTO student_marks (usn, marks) VALUES ('USN99', 55);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`student_db`.`student_marks`, CONSTRAINT `student_marks_ibfk_1` FOREIGN KEY (
`usn`) REFERENCES `student_details` (`usn`))
mysql> DELETE FROM student_details WHERE usn = 'USN01';
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`student_db`.`student_marks`, CONSTRAINT `student_marks_ibfk_1` FOREIGN K
EY (`usn`) REFERENCES `student_details` (`usn`))
mysql> UPDATE student_details SET usn = 'USN01_NEW' WHERE usn = 'USN01';
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`student_db`.`student_marks`, CONSTRAINT `student_marks_ibfk_1` FOREIGN K
EY (`usn`) REFERENCES `student_details` (`usn`))
mysql>
DELL


| performance_schema |
| sys |
4 rows in set (0.01 sec)
mysql> CREATE DATABASE student_db;
Query OK, 1 row affected (0.01 sec)
mysql> USE student_db;
Database changed
mysql> CREATE TABLE student_details (
-> usn VARCHAR(12) NOT NULL PRIMARY KEY,
-> student_name VARCHAR(100) NOT NULL,
-> course_name VARCHAR(100) NOT NULL
->);
Query OK, 0 rows affected (0.04 sec)
mysql> CREATE TABLE student_marks (
-> id INT AUTO_INCREMENT PRIMARY KEY,
-> usn VARCHAR(12) NOT NULL,
-> marks INT NOT NULL,
-> FOREIGN KEY (usn) REFERENCES student_details(usn)
->);
Query OK, 0 rows affected (0.05 sec)
mysql> INSERT INTO student_details (usn, student_name, course_name) VALUES
-> ('USN01', 'Anita Rao', 'B.E. Computer Science'),
-> ('USN02', 'Ravi Kumar', 'B.E. Electronics'),
-> ('USN03', 'Maya Singh', 'B.E. Mechanical');
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0
mysql> INSERT INTO student_marks (usn, marks) VALUES
-> ('USN01', 82),
-> ('USN02', 74);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>


Windows PowerShell
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0
mysql> INSERT INTO student_marks (usn, marks) VALUES
-> ('USN01', 82),
-> ('USN02', 74);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> SELECT * FROM student_details;
usn	student_name	course_name
USN01	Anita Rao	B.E. Computer Science
USN02	Ravi Kumar	B.E. Electronics
USN03	Maya Singh	B.E. Mechanical
3 rows in set (0.00 sec)
mysql> SELECT * FROM student_marks;
id	usn	marks
1	USN01	82
2	USN02	74
2 rows in set (0.00 sec)
mysql>


Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
PS C:\Program Files\MySQL> mysql --version
C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe Ver 8.0.44 for Win64 on x86_64 (MySQL Community Server - GPL)
PS C:\Program Files\MySQL> mysql -u root -p
Enter password: **********
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.0.44 MySQL Community Server - GPL
Copyright (c) 2000, 2025, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> show databases;
| Database |
| information_schema |
| mysql |
| performance_schema |
| sys |
4 rows in set (0.01 sec)
mysql> CREATE DATABASE student_db;
Query OK, 1 row affected (0.01 sec)
mysql> USE student_db;
Database changed
mysql> CREATE TABLE student_details (
-> usn VARCHAR(12) NOT NULL PRIMARY KEY,
-> student_name VARCHAR(100) NOT NULL,
-> course_name VARCHAR(100) NOT NULL


MySQL 8.0 Command Line Cli
ERROR 1046 (3D000): No database selected
mysql>
mysql> INSERT INTO Student (StudentName, DeptID, Marks) VALUES
-> ('Arun', 1, 85),
-> ('Neha', 2, 90),
-> ('Rahul', 1, 70),
-> ('Sita', 3, 88);
ERROR 1046 (3D000): No database selected
mysql> CREATE VIEW StudentInfo AS
-> SELECT s.StudentName, d.DeptName, s.Marks
-> FROM Student s
-> JOIN Department d ON s.DeptID = d.DeptID;
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM StudentInfo;
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Marks_Audit (
-> AuditID INT PRIMARY KEY AUTO_INCREMENT,
-> StudentID INT,
-> OldMarks INT,
-> NewMarks INT,
-> ChangedOn DATETIME DEFAULT CURRENT_TIMESTAMP
->);
ERROR 1046 (3D000): No database selected
mysql> DELIMITER $$
mysql> CREATE TRIGGER after_marks_update
-> AFTER UPDATE ON Student
-> FOR EACH ROW
-> BEGIN
-> INSERT INTO Marks_Audit (StudentID, OldMarks, NewMarks)
-> VALUES (OLD.StudentID, OLD.Marks, NEW.Marks);
-> END $$
ERROR 1046 (3D000): No database selected
mysql> DELIMITER ;
mysql> UPDATE Student SET Marks = 95 WHERE StudentID = 1;
ERROR 1046 (3D000): No database selected
mysql> SELECT * FROM Marks_Audit;
ERROR 1046 (3D000): No database selected
mysql> DROP VIEW StudentInfo;
ERROR 1046 (3D000): No database selected
mysql> DROP TRIGGER after_marks_update;
ERROR 1046 (3D000): No database selected